survlearning: A Unified R Interface for Survival Learning with Censored Data

Pedro Luiz Ramos
Pontificia Universidad Católica de Chile

Abstract

Failure-time prediction with covariates requires methods that account for censoring and distinguish risk ranking from probability estimation. Although several implementations are available in R, their fitting, hyperparameter, and prediction interfaces differ. In this paper, we present **survlearning**, a formula-based interface for fitting, tuning, and predicting with established survival learning methods. The package provides 23 method configurations, including parametric and penalized semiparametric accelerated failure time models, penalized Cox models, survival trees, random survival forests, Cox gradient boosting, XGBoost survival models, multi-task logistic regression, native neural Cox estimation with tied failures, and censored support vector regression. The neural Cox implementation uses analytic gradients in R, without a separate neural runtime. Linear and additive SVM implementations preserve the reference conditioned quadratic program through compact matrix algebra and explicit numerical checks. A common event-stratified cross-validation procedure uses Harrell's concordance index, with predictor encoding and standardization estimated within each training fold. Prediction capabilities are declared explicitly; a risk score is not converted into a survival distribution without an implemented statistical construction. We describe the estimation objectives, software design, and reproducible use of the interface. A controlled generated example exercises all 23 identifiers, and a historical reliability application fits 14 configurations to the complete training portion of 29,058 eligible hard drives with 718 subsequent failures, reconstructed from 29,072 public drive histories. Separate computational experiments report elapsed time, process memory, numerical equivalence, and held-out concordance. Additional interfaces provide permutation importance, accumulated local effects, conditional remaining-time summaries, covariate-dependent censoring weights, and resumable local parallel tuning. These operations retain explicit support and validation contracts. The examples illustrate the interface and its limitations, rather than establish a universal ordering of the methods. The software uses optional computational engines and is distributed under the GNU General Public License, version 3 or later.

Keywords: survival analysis, reliability, statistical learning, censored data, regularization, concordance, R.

1. Introduction

Reliability studies frequently involve predicting the time until failure of components operating under heterogeneous conditions. For a hard drive, the available information may include manufacturer, capacity, operating hours, temperature, and diagnostic attributes. The quantities of interest are not necessarily the same. Maintenance prioritization may require a ranking of

Software	Main organization	Relation to the present package
mlr3proba	Tasks, learners, distributions, pipelines, and resampling	Broader infrastructure; its website announces a transition to successor packages.
censored	Survival engines within the **parsnip** interface	Integrates with the wider **tidymodels** workflow.
survalis	Unified modeling, evaluation, and interpretation	Broader model and diagnostic scope.
SuperSurv	Cross-validated ensembles or selection of one best learner	Supports both modes; selecting one learner is not unique to the present package.
survlearning	Formula, grid, fit, and declared prediction types	Small base-style interface with optional engines and concordance tuning.

Table 1: Selected related software and the scope of the present interface. The comparison describes documented functionality, not an empirical comparison of predictive performance or runtime. Software information was checked in October 2026.

components, whereas resource planning may require a failure probability within a specified horizon. A conditional median failure time is another possible target. Since many components remain operational when follow-up ends, these quantities must be estimated in the presence of right censoring.

Classical failure-time models provide a well-established formulation for this problem (Lawless 2003; Kalbfleisch and Prentice 2002). Accelerated failure time (AFT) models relate covariates to the time scale, and the Cox model relates them to the hazard function (Cox 1972). Regularization permits shrinkage and variable selection within these formulations. Tree-based methods, boosting, and kernel methods provide additional ways to represent nonlinear effects and interactions. Their statistical assumptions and returned predictions, however, differ. A common interface should not conceal these differences.

In R (R Core Team 2026), established engines include **survival**, **glmnet**, **penAFT**, **rpart**, **randomForestSRC**, **gbm**, **survivalsvm**, **xgboost**, and **MTLR**. These implementations make a broad set of methods accessible, but their use in a single analysis requires several decisions. A penalty parameter must be supplied according to the appropriate objective; a predicted log-time must be distinguished from a log-hazard score; factors and standardization must be handled consistently; and tuning must not use the external test sample. For teaching and small comparative studies, repeatedly implementing these operations can obscure the statistical problem.

Unified survival learning is not a new software objective. The **mlr3proba** framework provides survival tasks, learners, prediction objects, and evaluation within the **mlr3** ecosystem (Sonabend, Király, Bender, Bischl, and Lang 2021). The **censored** package extends the **parsnip** interface to censored regression (Hvitfeldt and Frick 2026). More recent packages include **survalis**, with modeling and interpretation facilities, and **SuperSurv**, with a cross-validated ensemble and best-learner selection modes (El Badisy 2026; Lyu 2026; SuperSurv Authors 2026). In Python, **scikit-survival** provides an interface compatible with **scikit-learn** (Pölsterl 2020). These projects have broader capabilities beyond the scope considered here. Table 1 places the present interface within this setting.

In this paper, we present **survlearning**, a small interface organized around `surv_fit()`, `surv_tune()`, `surv_predict()`, and `surv_score()`. The main objective is to make the fitting and evaluation procedure explicit while retaining the statistical meaning of each method. The contribution comprises the interface, its prediction conventions, and reproducible numerical implementations of two computationally relevant families. Native Cox neural estimation avoids an external neural runtime and handles event-time ties explicitly. The SVM implementation exploits the structure of the reference conditioned quadratic program without changing the selected kernel or removing observations. We do not propose a new statistical loss or a generalization guarantee. Established engines retain their authorship and licenses.

Three design choices are central. First, a larger returned risk score always indicates greater risk, but its numerical scale remains method-specific. Second, preprocessing learned by the wrapper is estimated again within each training fold. Third, survival curves and median times are available only where their construction has been implemented. For example, the current support vector machine (SVM) interface returns a risk score, not an automatically calibrated failure probability. These choices reduce interface variation without asserting equivalence between statistical models.

The remainder of this paper is organized as follows. Section 2 introduces the observed data, prediction targets, and concordance criterion. Section 3 presents the implemented methods and their estimation objectives. Section 4 describes the software design and tuning procedure. Sections 5 and 6 present reproducible examples. Section 7 illustrates interpretation, remaining duration, conditional assessment, and resumable tuning. Section 8 reports numerical agreement and controlled computational comparisons. Section 9 describes the replication material. Finally, Section 10 discusses the scope and limitations of the current version.

2. Observed data and prediction targets

2.1. Right censoring and the survival function

Let $T_i > 0$ denote the failure time, $C_i > 0$ the censoring time, and $x_i \in \mathbb{R}^p$ the covariate vector for unit i. We observe

$$Y_i = \min(T_i, C_i), \quad \delta_i = \mathbb{I}(T_i \le C_i), \quad \mathcal{D} = \{(Y_i, \delta_i, x_i) : i = 1, \dots, n\}. \tag{1}$$

The standard likelihood interpretation assumes conditional independent censoring, $T_i \perp C_i \mid x_i$, and independent observational units. Neither assumption follows merely from using a survival learning method. In particular, nonfailure removal from a reliability cohort may be informative. The conditional survival, hazard, and cumulative hazard functions are given by

$$\begin{aligned} S(t \mid x) &= \Pr(T > t \mid x), \\ h(t \mid x) &= \lim_{a \downarrow 0} \frac{\Pr(t \le T < t + a \mid T \ge t, x)}{a}, \qquad H(t \mid x) = \int_0^t h(u \mid x)\, du. \end{aligned} \tag{2}$$

For a continuous failure time, $S(t \mid x) = \exp\{-H(t \mid x)\}$. Ignoring the censoring distribution under the stated assumption, the likelihood contribution is

$$L_i = f(Y_i \mid x_i)^{\delta_i} S(Y_i \mid x_i)^{1-\delta_i}. \tag{3}$$

Thus, a censored observation contributes a survival probability, not a density at an observed failure time. Replacing every censored time by an event time does not preserve Equation 3.

For descriptive analysis, let $t_1 < \cdots < t_J$ be the distinct observed event times, d_j the number of events at t_j, and n_j the number at risk just before t_j. The Kaplan–Meier estimator is

$$\widehat{S}_{\mathrm{KM}}(t) = \prod_{j:t_j \le t} \left(1 - \frac{d_j}{n_j}\right). \tag{4}$$

Equation 4 can be applied within covariate groups to describe observed failure behavior (Kaplan and Meier 1958). Such curves do not by themselves estimate a causal covariate effect or assess calibration of individual predictions.

2.2. Risk scores, probabilities, and time summaries

A risk score $r(x)$ is a scalar used to order units. The package adopts the convention that $r(x_a) > r(x_b)$ indicates greater predicted risk for unit a. This convention does not imply that $r(x)$ lies between zero and one. A Cox linear predictor, forest mortality, and negative predicted time can all satisfy this convention while having unrelated scales. When a survival function is available, the failure probability at horizon t_0 is $F(t_0 \mid x) = 1 - S(t_0 \mid x)$. The conditional median and restricted mean are given by

$$m(x) = \inf\{t : S(t \mid x) \le 1/2\}, \qquad \mu_\tau(x) = \int_0^\tau S(t \mid x)\, dt. \tag{5}$$

The horizon τ must be specified for the restricted mean. If an estimated curve remains above $1/2$ over the observed support, a median cannot be obtained without a tail assumption. A missing nonparametric median is not evidence of an infinite lifetime or a cured component. The current package does not fit cure models.

2.3. Harrell concordance

For distinct observed times, define the comparable ordered pairs by $\mathcal{P} = \{(i,j) : \delta_i = 1, Y_i < Y_j\}$. With a larger score indicating greater risk, Harrell's concordance index is estimated by

$$\widehat{C} = \frac{\sum_{(i,j)\in\mathcal{P}} \left[\mathbb{I}\{r(x_i) > r(x_j)\} + \frac{1}{2}\mathbb{I}\{r(x_i) = r(x_j)\}\right]}{|\mathcal{P}|}. \tag{6}$$

The formula assigns half credit to tied scores. With tied observed times, the implementation uses the event-time and censoring-time conventions of `survival::concordance()`, rather than adding numerical perturbations. The actual computation uses `reverse = TRUE`, since larger scores indicate shorter survival, and the default Harrell time weighting. The performance is evaluated in terms of this discrimination criterion (Harrell, Califf, Pryor, Lee, and Rosati 1982; Harrell, Lee, and Mark 1996). A value near one indicates concordant ordering among comparable pairs, and a value near 1/2 indicates little ordering information. Harrell concordance depends on the observed censoring pattern. It is not a calibration criterion, a failure detection rate, or the probability that a reported median is correct. These examples select models using Harrell concordance only. The current evaluation interface also provides truncated Uno concordance, time-dependent AUC, Brier scores, and calibration diagnostics

Identifier	Engine	Prediction types
`aft`	**survival**	Risk, median, survival
`aft_lasso`, `aft_ridge`	**penAFT**	Risk
`aft_elasticnet`	**penAFT**	Risk
`cox`	**survival**	Risk, survival
`cox_lasso`, `cox_ridge`	**glmnet**	Risk, survival
`cox_elasticnet`	**glmnet**	Risk, survival
`tree`	**rpart**	Risk
`rsf`	**randomForestSRC**	Risk, survival
`boosting`	**gbm**	Risk
`xgboost_cox`	**xgboost**	Risk, survival
`xgboost_aft`	**xgboost**	Risk, median, survival
`mtlr`	**MTLR**	Risk, survival
`svm`	Reference-compatible compact QP	Risk
`km`, `cox_spline`	**survival**	Risk, survival
`flexsurv`	**flexsurv**	Risk, survival
`coxboost`	**CoxBoost**	Risk, survival
`ranger`, `aorsf`	**ranger**, **aorsf**	Risk, survival
`neural_cox`	Native R	Risk, survival
`neural_discrete`	**nnet**	Risk, survival

Table 2: Methods and prediction capabilities in version 0.2.5. Risk scores increase with risk, but their scales are not comparable across methods. An unavailable prediction type generates an error.

when their required prediction types and censoring support exist; it does not silently replace the criterion requested by the user.

3. Statistical learning methods

We describe the estimation objectives because a common calling sequence does not make the methods interchangeable. The 23 identifiers correspond to configurations within several families, rather than 23 unrelated algorithms. Table 2 links models, engines, and implemented prediction types.

3.1. Parametric accelerated failure time models

The AFT formulation relates the logarithm of failure time to covariates:

$$\log T_i = \beta_0 + x_i^\top \beta + \sigma\varepsilon_i, \qquad \sigma > 0. \tag{7}$$

Let $\eta_i = \beta_0 + x_i^\top \beta$ and $z_i = (\log Y_i - \eta_i)/\sigma$. For error density f_ε and survival function S_ε, the log-likelihood is

$$\ell = \sum_{i=1}^{n} \left[\delta_i \{\log f_\varepsilon(z_i) - \log\sigma - \log Y_i\} + (1-\delta_i)\log S_\varepsilon(z_i)\right]. \tag{8}$$

The estimates are obtained by numerical maximization using `survival::survreg()` (Therneau 2026; Therneau and Grambsch 2000). Normal, logistic, and minimum extreme-value

errors give log-normal, log-logistic, and Weibull failure times, respectively. Under Equation 7, a one-unit increase in covariate j multiplies each conditional time quantile by $\exp(\beta_j)$, holding other covariates fixed. This time-ratio interpretation differs from a hazard ratio.

The fitted survival function is $\widehat{S}(t \mid x) = S_\varepsilon\{(\log t - \widehat{\eta}(x))/\widehat{\sigma}\}$. Its median is $\widehat{m}(x) = \exp\{\widehat{\eta}(x) + \widehat{\sigma} q_{\varepsilon,0.5}\}$, where $q_{\varepsilon,0.5}$ is the error median. The default risk score is $-\widehat{\eta}(x)$. Parametric curves can be evaluated beyond the largest observed follow-up, but this is model-based extrapolation, not additional information supplied by censored observations.

3.2. Penalized semiparametric accelerated failure time models

The penalized AFT implementation does not maximize a penalized version of Equation 8. It uses a rank-based Gehan objective, without specifying the error distribution. Let $e_i(\beta) = \log Y_i - x_i^\top \beta$ and $(a)_+ = \max(a, 0)$. A normalized Gehan loss is

$$L_G(\beta) = \frac{1}{n^2} \sum_{i=1}^{n} \sum_{j=1}^{n} \delta_i \left(e_j(\beta) - e_i(\beta)\right)_+ . \tag{9}$$

An intercept cancels from the residual differences and is not identified by this criterion. For $\lambda > 0$ and $0 \le \alpha \le 1$, define

$$P_{\lambda,\alpha}(\beta) = \lambda \left\{ \alpha \|\beta\|_1 + \frac{1-\alpha}{2} \|\beta\|_2^2 \right\}, \quad \widehat{\beta} = \arg\min_\beta \{L_G(\beta) + P_{\lambda,\alpha}(\beta)\}. \tag{10}$$

The choices $\alpha = 1$ and $\alpha = 0$ give LASSO and ridge penalties; intermediate values give the elastic net (Zou and Hastie 2005). LASSO can set coefficients to zero, whereas ridge shrinks them without imposing sparsity in the usual continuous setting.

The **penAFT** engine uses an alternating direction method of multipliers for this nonsmooth optimization problem (Suder and Molstad 2022; Molstad and Suder 2025). The wrapper supplies training-standardized predictors and disables a second scaling operation in the engine. The three penalized AFT identifiers differ in α. Their returned score is $-x^\top \widehat{\beta}$. Since an error distribution and its location are not estimated by this adapter, survival probabilities and median times are not exposed. The numerical meaning of λ depends on the loss normalization and engine; it should not be transferred unchanged to a different family.

3.3. Cox models and regularization

The proportional hazards formulation is

$$h(t \mid x) = h_0(t) \exp(x^\top \beta), \quad S(t \mid x) = \exp\{-H_0(t) \exp(x^\top \beta)\}. \tag{11}$$

For distinct failure times, the partial log-likelihood is given by

$$\ell_P(\beta) = \sum_{i:\delta_i=1} \left[x_i^\top \beta - \log \left\{ \sum_{j \in R(Y_i)} \exp(x_j^\top \beta) \right\} \right], \quad R(t) = \{j : Y_j \ge t\}. \tag{12}$$

The baseline hazard cancels from Equation 12; it need not be specified to estimate β. The ordinary adapter uses `coxph()` with the Efron approximation for tied failures. A coefficient gives a conditional hazard ratio $\exp(\beta_j)$ under the model, not a constant ratio of failure probabilities.

For penalized Cox estimation, the objective has the form

$$\widehat{\beta} = \arg\min_{\beta} \{-a_n \ell_P(\beta) + P_{\lambda,\alpha}(\beta)\}, \tag{13}$$

where $a_n > 0$ denotes the loss normalization used by the engine. The **glmnet** implementation uses coordinate-descent procedures and its own training standardization (Simon, Friedman, Hastie, and Tibshirani 2011; Friedman, Hastie, and Tibshirani 2010). The wrapper explicitly requests Breslow ties for these fits. Consequently, an ordinary Cox fit and a weakly penalized fit can differ because of the tie convention as well as the penalty. The identifiers ending in `lasso`, `ridge`, and `elasticnet` correspond to $\alpha = 1$, 0, and an adjustable value, respectively. For a fitted risk predictor $\widehat{\eta}(x)$, a Breslow baseline is

$$\widehat{H}_0(t) = \sum_{j:t_j \le t} \frac{d_j}{\sum_{i \in R(t_j)} \exp\{\widehat{\eta}(x_i)\}}, \quad \widehat{S}(t \mid x) = \exp\{-\widehat{H}_0(t) \exp\{\widehat{\eta}(x)\}\}. \tag{14}$$

The ordinary and penalized adapters use their native baseline and prediction routines. The score is $\widehat{\eta}(x)$. Proportional hazards remains an assumption after regularization; a penalty does not remove it.

3.4. Native neural Cox estimation

The neural Cox formulation replaces the linear predictor in Equation 11 by a neural network (Katzman, Shaham, Cloninger, Bates, Jiang, and Kluger 2018). This changes the representation of covariate effects, not the proportional hazards assumption. We consider a single hidden layer with m units,

$$z_i = \tanh(W^\top \widetilde{x}_i + b), \qquad \eta_i = f_\theta(x_i) = v^\top z_i + c, \qquad h(t \mid x_i) = h_0(t) \exp(\eta_i), \tag{15}$$

where $\widetilde{x}_i$ is standardized using training means and standard deviations. All weights and biases are included in θ. A positive difference in network outputs represents a larger log hazard ratio; the network output is neither a probability nor a predicted failure time. Its additive location is absorbed by the baseline hazard. Individual network weights are not assigned hazard-ratio interpretations; inference concerns the fitted risk function and its associated survival curves.

Tied failures and the baseline

Let t_g denote a distinct observed failure time, D_g its failure set, $d_g = |D_g|$, and $R_g = \{i : Y_i \ge t_g\}$. Define $S_g = \sum_{i \in R_g} \exp(\eta_i)$ and $E_g = \sum_{i \in D_g} \exp(\eta_i)$. With $N_E = \sum_i \delta_i$, the penalized objective used by the native implementation is

$$\mathcal{L}(\theta) = \frac{1}{N_E} \left[\sum_g \sum_{l=0}^{d_g - 1} \log\{S_g - (l/d_g) E_g\} - \sum_{i:\delta_i = 1} \eta_i \right] + \frac{\lambda_N}{2} \|\theta\|^2. \tag{16}$$

This is Efron's approximation (Efron 1977). The complete risk set includes failures and censorings at t_g; only failure mass is subtracted in E_g. Thus a censoring tied with a failure is not treated as a failure or removed before that event's denominator. Breslow is obtained by replacing the inner sum by $d_g \log S_g$. Without tied failures the two expressions coincide. Neither expression is the exact discrete partial likelihood, and no numerical perturbation is added to the times.

After estimating θ, we use the matching baseline increments

$$\Delta\widehat{H}_0(t_g) = \begin{cases} \sum_{l=0}^{d_g-1} \{\widehat{S}_g - (l/d_g)\widehat{E}_g\}^{-1}, & \text{Efron}, \\ d_g/\widehat{S}_g, & \text{Breslow}. \end{cases} \tag{17}$$

The cumulative sum of these increments and $\widehat{S}(t \mid x) = \exp[-\widehat{H}_0(t)\exp\{f_{\widehat{\theta}}(x)\}]$ provide survival curves. The baseline is estimated from training outcomes only, including the full-training refit after tuning. Internally it is stored on a centered logarithmic scale to avoid overflow; this centering is compensated in the network risk and does not change the modeled curve.

Analytic derivatives and training

For Efron, write

$$A_g = \sum_{l=0}^{d_g-1} \frac{1}{S_g - (l/d_g)E_g}, \qquad B_g = \sum_{l=0}^{d_g-1} \frac{l/d_g}{S_g - (l/d_g)E_g}. \tag{18}$$

The derivative of the unpenalized part of Equation 16 with respect to a log risk is given by

$$\frac{\partial \mathcal{L}_{\text{data}}}{\partial \eta_i} = \frac{1}{N_E}\left[\exp(\eta_i)\sum_{g:i\in R_g} A_g - \delta_i - \delta_i \exp(\eta_i) B_{g(i)}\right], \tag{19}$$

where $g(i)$ is the failure group of an observed event; its term is zero for a censored row. Under Breslow, $A_g = d_g/S_g$ and $B_g = 0$. Ordering and group membership are prepared once. Cumulative sums evaluate the risk-set contributions without constructing an $n \times n$ array. Log-sum-exp calculations handle wide ranges of risk values. Efficient grouped Cox calculations with both Breslow and Efron ties have also been studied by Yang, Abraham, Kim, Smirnov, Ruan, Haibe-Kains, and Tibshirani (2022). The present contribution is their use within a native R workflow with explicit prediction contracts, not a claim that efficient tied-risk-set evaluation is new.

Let $e_i = \partial \mathcal{L}_{\text{data}}/\partial \eta_i$ and $a_i = e_i v \odot (1 - z_i \odot z_i)$. Backpropagation gives

$$\begin{aligned} \nabla_W \mathcal{L} &= \sum_i \widetilde{x}_i a_i^\top + \lambda_N W, & \nabla_b \mathcal{L} &= \sum_i a_i + \lambda_N b, \\ \nabla_v \mathcal{L} &= \sum_i e_i z_i + \lambda_N v, & \partial_c \mathcal{L} &= \sum_i e_i + \lambda_N c. \end{aligned} \tag{20}$$

These expressions are evaluated using ordinary R matrix operations. The penalty includes biases as well as connection weights. Full-batch Adam uses the complete training gradient at every update (Kingma and Ba 2015):

$$\begin{gathered} m_k = \beta_1 m_{k-1} + (1-\beta_1) g_k, \qquad v_k = \beta_2 v_{k-1} + (1-\beta_2) g_k^2, \\ \theta_k = \theta_{k-1} - \rho \frac{m_k/(1-\beta_1^k)}{\sqrt{v_k/(1-\beta_2^k)} + \epsilon_A}. \end{gathered} \tag{21}$$

Squares, roots, and division in Equation 21 are componentwise. The implementation uses $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon_A = 10^{-8}$. Learning rate ρ, hidden size, penalty, and epoch budget are explicit parameters. Fixed epochs do not certify convergence to a stationary point or a global optimum. The final loss, penalty, maximum absolute gradient, tie counts, and sample size remain available through `surv_diagnostics()`. No test-based early stopping is performed.

The identifier `neural_cox` requires neither **torch**, **survdnn**, **reticulate**, nor a Python environment. Its saved parameters are ordinary R arrays. It follows the same neural proportional hazards family as DeepSurv, but the present single-layer architecture is not described as an exact reproduction of its original architecture or tuning protocol. Numerical equivalence and predictive performance are assessed separately in Section 8.

3.5. Survival trees and random survival forests

A survival tree partitions the covariate space into disjoint terminal regions $A_1, \ldots, A_L$. Its prediction is constant within each terminal region. The `tree` adapter uses `rpart(method = "exp")`, an exponential-deviance construction (Breiman, Friedman, Olshen, and Stone 1984; Therneau and Atkinson 2026). It is not a log-rank tree. Let $\widetilde{H}_0(Y_i)$ denote the transformed exposure obtained from a training baseline. Within a node A, the unshrunk relative-rate estimate maximizes

$$\ell_A(\theta) = \sum_{i:x_i \in A} \{\delta_i \log \theta - \theta \widetilde{H}_0(Y_i)\}, \quad \widehat{\theta}_A = \frac{\sum_{i:x_i \in A} \delta_i}{\sum_{i:x_i \in A} \widetilde{H}_0(Y_i)}. \tag{22}$$

Terms not depending on θ are omitted. Equation 22 describes the unshrunk likelihood calculation. The engine additionally uses its default rate-shrinkage setting, `shrink = 1`, so returned leaf rates need not equal the unshrunk ratio. Splits improve the corresponding deviance, with complexity and node-size controls restricting the tree. If $D(\mathcal{T})$ is its deviance, cost-complexity pruning compares criteria of the form $D(\mathcal{T}) + \kappa|\mathcal{T}|$. The adapter returns the native node risk score; it does not add a nodewise survival-curve estimator.

Random survival forests (RSF) combine trees fitted to bootstrap samples (Ishwaran, Kogalur, Blackstone, and Lauer 2008). Only a random subset of predictors is considered at each node. For a candidate split, a standardized log-rank statistic is

$$Z = \frac{\sum_j (d_{Lj} - n_{Lj} d_j / n_j)}{\left\{ \sum_{j: n_j > 1} \dfrac{n_{Lj} n_{Rj} d_j (n_j - d_j)}{n_j^2 (n_j - 1)} \right\}^{1/2}}. \tag{23}$$

Here n_{Lj} and n_{Rj} are the risk-set counts in the proposed children. The **randomForestSRC** adapter requests log-rank splitting; `mtry`, minimum node size, and candidate split points control the construction (Ishwaran and Kogalur 2026). For tree b and the terminal node containing x, a cumulative hazard is $\widehat{H}_b(t \mid x) = \sum_{t_j \le t} d_{bj}(x)/n_{bj}(x)$. Terminal estimates are averaged over trees:

$$\widehat{H}_{\mathrm{RSF}}(t \mid x) = B^{-1} \sum_b \widehat{H}_b(t \mid x), \quad \widehat{S}_{\mathrm{RSF}}(t \mid x) = B^{-1} \sum_b \widehat{S}_b(t \mid x). \tag{24}$$

The two averages need not satisfy an exact exponential identity. The native mortality score sums ensemble cumulative hazard over the training event-time grid. It orders units but is not a failure probability. Out-of-bag observations support internal forest diagnostics; here hyperparameters are selected using the same held-out folds as for the other engines.

3.6. Gradient boosting with a Cox objective

For tree learners g_m, a boosted predictor after M stages is

$$f_M(x) = f_0(x) + \nu \sum_{m=1}^{M} \rho_m g_m(x), \qquad \nu > 0. \tag{25}$$

Considering the negative partial log-likelihood $\mathcal{L}(f) = -\ell_P(f)$, a negative gradient at stage m is

$$u_i^{(m)} = \delta_i - \sum_{k:\delta_k=1,\, Y_k \le Y_i} \frac{\exp\{f_{m-1}(x_i)\}}{\sum_{j \in R(Y_k)} \exp\{f_{m-1}(x_j)\}}. \tag{26}$$

Equation 26 is written for distinct event times. A weak tree approximates the gradient and a loss-based step updates the predictor. The stages, depth, learning rate, and subsampling fraction jointly control the fit (Friedman 2001). The `boosting` identifier uses `gbm(distribution = "coxph")` and returns its log-risk predictor (Ridgeway and Developers 2026). This is gradient boosting with tree learners, not componentwise likelihood-based Cox boosting (Binder and Schumacher 2008). A nonlinear predictor in Equation 11 still implies proportional hazards without a time-dependent predictor. The current adapter exposes a risk score only; no baseline hazard or calibration model is silently added.

3.7. XGBoost survival objectives

The **xgboost** engine builds regularized additive trees using gradient and curvature information (Chen and Guestrin 2016; Chen, He, Benesty, Khotilovich, Tang, Cho, Chen, Mitchell, Cano, Zhou, Li, Xie, Lin, Geng, Li, Yuan, and Cortes 2026). For tree $g_m(x) = w_{m,q_m(x)}$, with leaf assignment q_m, a stagewise objective has the form

$$\mathcal{J}_m = \mathcal{L}(f_{m-1} + g_m) + \Omega(g_m), \quad \Omega(g_m) = \gamma L_m + \frac{\lambda_w}{2} \sum_{\ell=1}^{L_m} w_{m\ell}^2. \tag{27}$$

Second-order approximations guide split and leaf calculations. The wrapper uses histogram-based tree construction and exposes tree count, depth, learning rate, minimum child weight, and leaf penalty. It does not expose every native option.

For `xgboost_cox`, the loss is the Cox loss and the margin is a log-risk predictor. Internally, a failure has label $+Y_i$ and a censored observation has label $-Y_i$. The adapter estimates the Breslow baseline in Equation 14 using training outcomes only, combining tied event counts. For numerical evaluation, store the training center $c = \min_i \widehat{\eta}_i/2 + \max_i \widehat{\eta}_i/2$ and write $z_i = \widehat{\eta}_i - c$. The halved-endpoint calculation avoids overflowing their sum. Let $a_g = \mathrm{LSE}\{z_i : i \in R_g\}$, where $\mathrm{LSE}(u_1, \ldots, u_k) = m + \log \sum_j \exp(u_j - m)$ and $m = \max_j u_j$. We compute

$$\ell_g = \log d_g - a_g, \qquad L(t) = \mathrm{LSE}\{\ell_g : t_g \le t\}, \qquad \widehat{S}(t \mid x) = \exp[-\exp\{\widehat{\eta}(x) - c + L(t)\}]. \tag{28}$$

A time with no events has $\ell_g = -\infty$; before the first event, $L(t) = -\infty$ and survival is one. Both risk-set sums and cumulative increments are evaluated in log space. Centering also prevents a very large common margin from erasing small log-count corrections. It does not recover information already lost in the input margins. A single global centering followed by ordinary exponentiation is insufficient when the largest margin leaves a later risk set. Equation 28 preserves the Breslow estimator without clipping margins or changing the Cox objective.

For `xgboost_aft`, a nonlinear predictor replaces the linear location in Equation 7. The response is supplied through bounds:

$$[L_i, U_i] = \begin{cases} [Y_i, Y_i], & \delta_i = 1, \\ [Y_i, \infty), & \delta_i = 0. \end{cases} \tag{29}$$

The loss uses a density for exact events and a survival probability for censoring (Barnwal, Cho, and Hocking 2022). The error distribution is normal, logistic, or minimum extreme value; its scale is a tuning parameter, not an additional parameter estimated automatically by the wrapper. For log-time location $f(x)$, the score is $-f(x)$ and

$$\widehat{S}(t \mid x) = S_\varepsilon\left(\frac{\log t - f(x)}{\sigma}\right), \quad \widehat{m}(x) = \exp\{f(x) + \sigma q_{\varepsilon,0.5}\}. \tag{30}$$

For normal and logistic errors, $q_{\varepsilon,0.5} = 0$. For minimum extreme-value errors, $q_{\varepsilon,0.5} = \log(\log 2)$. Exponentiating the margin alone does not give the median in the latter case.

3.8. Multi-task logistic regression

Multi-task logistic regression (MTLR) estimates a distribution over ordered failure-time intervals (Yu, Greiner, Lin, and Baracos 2011). Let $0 = \tau_0 < \cdots < \tau_K$ be a training-defined grid, with a final interval beyond τ_K. Define $a_k(x) = \sum_{j=k}^{K}(b_j + x^\top \theta_j)$ for $k = 1, \ldots, K+1$, with $a_{K+1}(x) = 0$. The interval probabilities are

$$p_k(x) = \frac{\exp\{a_k(x)\}}{\sum_{\ell=1}^{K+1}\exp\{a_\ell(x)\}}, \quad S(\tau_j \mid x) = \sum_{k=j+1}^{K+1} p_k(x). \tag{31}$$

The construction restricts possible survival-status sequences; it is not a collection of unrelated binary regressions. For an event in interval k_i, the contribution is $p_{k_i}(x_i)$; for censoring at boundary τ_{j_i}, it is $\sum_{k>j_i} p_k(x_i)$. A representative penalized criterion is

$$-\sum_i \left\{ \delta_i \log p_{k_i}(x_i) + (1-\delta_i)\log \sum_{k>j_i} p_k(x_i) \right\} + \frac{\lambda}{2}\sum_{j=1}^{K}\|\theta_j\|_2^2. \tag{32}$$

The native engine determines encoding between knots. The original formulation also considered adjacent-parameter smoothing; **MTLR** exposes the coefficient penalty `C1`, named `lambda` by the wrapper (Haider 2026). Its optimization uses L-BFGS-B, with an initial fitting stage before the censored criterion. A global convexity guarantee should not be inferred from the uncensored multinomial representation.

Native interpolated survival probabilities are returned. For concordance, the adapter uses $r_{\mathrm{MTLR}}(x) = -\widehat{\mu}_\tau(x)$, with $\tau = \max_{i\in\mathcal{D}_{\mathrm{train}}} Y_i$. The integral in Equation 5 is approximated by

the trapezoidal rule on 101 equally spaced points together with learned knots. The horizon and grid are recomputed within each training fold. This scalarization does not impose proportional hazards or extrapolate a full mean. When curves cross, its ordering need not agree with risk at every horizon. Full-mean and median tail extrapolations are not exposed.

3.9. Support vector regression with censoring

The `svm` identifier implements the regression approach described for **survivalsvm**, not its ranking or hybrid alternatives (Fouodo, König, Weihs, Ziegler, and Wright 2018). Let $g(x) = b + \langle w, \phi(x) \rangle$ be a time score in a feature space. The censored regression objective is

$$\min_{w,b} \frac{1}{2}\|w\|^2 + \gamma \sum_i \left[(Y_i - g(x_i))_+ + \delta_i \, (g(x_i) - Y_i)_+ \right]. \tag{33}$$

For an event, the terms give absolute error. For a censored observation, a prediction above Y_i is not penalized: failure is known only to exceed Y_i. Equivalently, nonnegative slacks satisfy $Y_i - g(x_i) \le \xi_i$ for every observation and $g(x_i) - Y_i \le \xi_i^*$ for events. This distinguishes the objective from ordinary regression on recorded times.

The kernel matrix has entries $K_{ij} = \langle \phi(x_i), \phi(x_j) \rangle$. The adapter offers the native linear and additive kernels:

$$K_{\text{lin}}(x, z) = x^\top z, \qquad K_{\text{add}}(x, z) = \sum_{j=1}^{p} \left(1 - \frac{|x_j - z_j|}{R_j} \right), \quad R_j = \max_{i \in \mathcal{D}_{\text{train}}} x_{ij} - \min_{i \in \mathcal{D}_{\text{train}}} x_{ij}. \tag{34}$$

The additive expression describes numeric encoded predictors, not a radial basis kernel. Constant design columns are removed and remaining predictors are standardized using training means and standard deviations. The wrapper maps `cost` to the reference `gamma.mu`. Kernel evaluation is native R; **survivalsvm** is used for reference checks, not required for fitting or prediction. For compatibility with the reference implementation, additive prediction recomputes ranges on the support rows retained after rounding. Those ranges may differ from the full-training ranges in Equation 34, and the difference can affect ordering, not only the intercept. This convention is retained and audited rather than silently replaced; Appendix A gives the details. The returned score is $-\widehat{g}(x)$, not a calibrated median or probability. Faster survival SVM formulations exist (Pölsterl, Navab, and Katouzian 2015); replacing the loss by a ranking loss, however, would not be an optimization of the same estimator.

The computational target includes the reference positive-definite conditioning, support-coefficient rounding, and intercept convention. Compact solvers represent the same conditioned quadratic program, not the unconditioned textbook dual. Linear factors exploit the encoded predictor span; additive kernel products use sorted prefix sums and an adaptively verified eigensystem. Exact duplicate groups retain their full multiplicities. No training rows are removed. Clarabel and OSQP use a sparse augmented program; the native Newton path eliminates box constraints and checks an independent primal–dual certificate. Appendix A gives the complete construction, equivalence conditions, numerical thresholds, and fold-local workspace reuse.

3.10. Additional reference and learning configurations

The `km` identifier supplies the covariate-free benchmark in Equation 4; its constant risk score gives concordance 1/2 when comparable pairs exist. For `cox_spline`, the Cox linear predic-

tor is replaced by $B_d(x)^\top\beta$, where natural-spline bases are learned from training numeric predictors, with factors retained. Estimation still uses the Cox partial likelihood. The `flexsurv` adapter uses a flexible parametric spline model; on the hazard scale used here, $\log H(t \mid x) = s(\log t; \gamma) + x^\top\beta$. Likelihood contributions follow Equation 3 (Jackson 2016). Increasing spline flexibility does not remove the proportional-hazards restriction on that scale.

The `coxboost` adapter uses componentwise likelihood-based boosting (Binder and Schumacher 2008), rather than the regression-tree gradient boosting described above. At each step, a coordinate update is selected by its penalized partial-likelihood improvement and added to the current predictor; the number of steps controls regularization. The `ranger` and `aorsf` adapters provide alternative survival forests. The former uses axis-aligned splits; the latter searches oblique splits of the form $x^\top\beta \le c$ (Wright and Ziegler 2017; Jaeger, Welden, Lenoir, and Pajewski 2022; Jaeger, Welden, Lenoir, Speiser, Segar, Pandey, and Pajewski 2023). They retain their native ensemble hazard/survival constructions rather than replacing them by a Cox baseline.

Finally, `neural_discrete` uses a single-hidden-layer **nnet** model for person-period Bernoulli responses, following the partial logistic approach (Biganzoli, Boracchi, Mariani, and Marubini 1998; Venables and Ripley 2002). If $h_k(x)$ is the conditional probability of an event in interval k, then

$$S(\tau_j \mid x) = \prod_{k \le j} \{1 - h_k(x)\}, \quad \ell = \sum_{(i,k)\in\mathcal{O}} \{d_{ik} \log h_k(x_i) + (1 - d_{ik}) \log[1 - h_k(x_i)]\}. \tag{35}$$

The set $\mathcal{O}$ contains periods with observed event/survival status. For a censored unit, only completed intervals contribute; an incomplete censoring interval is not labeled as an observed nonevent. Coarsening loses partial-interval information, so this is not asserted to be the continuous-time likelihood. Its grid is learned from training, and prediction returns stepwise curves on that grid. This formulation differs from the native continuous-time neural Cox model in Section 3.4.

4. Software design and use

4.1. Installation, method discovery, and fitting

The package has a standard R source structure, with code in `R/`, help files in `man/`, examples in `inst/examples/`, and tests in `tests/testthat/`. Mandatory imports are **stats**, **survival**, **graphics**, **utils**, **parallel**, **tools**, and **withr** (Hester, Henry, Muller, Ushey, Wickham, and Chang 2024). Apart from the recommended **survival** package and the small RNG-scoping dependency **withr**, these ship with R. No neural runtime is installed. Model-specific engines are optional dependencies: an ordinary Cox example does not require installing the forest or kernel engines. The tested engine versions are recorded in the replication material.

The source is available at `https://github.com/ramospedroluiz/survlearning`. From the directory containing the source archive, one uses

```
R> install.packages(c("survival", "withr"))
R> install.packages("survlearning_0.2.5.tar.gz",
+    repos = NULL, type = "source")
R> library("survlearning")
R> surv_methods()
```

The method catalogue reports the engine, supported prediction types, and whether the engine is installed. A missing optional dependency produces an informative error when fitting is requested.

The common fitting signature is

```
surv_fit(formula, data, method = "cox", params = list(),
         seed = NULL, impute = "none")
```

The response must be a right-censored `Surv(time, event)` object, with positive times and event indicators zero or one. Formulas with factors, pointwise transformations, and interactions are supported. A transformation such as `log1p(hours)` is evaluated using the stored formula; it does not estimate an imputation or scaling parameter from new rows. By default, missing values are rejected; explicit `impute = "median"` estimates numeric medians and factor modes from training only. Outcomes are never imputed. New factor levels and unsupported formula specials are rejected. Dot formulas are resolved against the original training response. This also applies to censoring models before reversing the event indicator, so the original time and status columns do not become predictors through `.`. Counting-process responses, offsets, strata, clusters, frailties, and time-dependent effects are outside the current interface.

The returned `survlearning_fit` object stores the formula, method, resolved parameters, design specification, native engine, sample size, number of events, maximum follow-up, seed, and fitting time. The native object is retained in `object$engine` for inspection. Its direct use requires the encoded design expected by that engine; bypassing the wrapper can also bypass its prediction conventions.

4.2. Preprocessing and parameter translation

The design specification is learned from the training model frame. It stores terms, factor levels, contrasts, retained design columns, and required scaling quantities. Constant columns are removed. New rows are encoded using this specification, not a design estimated afresh from the prediction sample. The SVM, Gehan AFT, MTLR, and native neural Cox adapters use training means and standard deviations; **glmnet** performs its own training standardization.

There is no hidden imputation. If requested, its parameters are relearned inside each training fold. Missing-data handling remains part of the analysis; median/mode imputation does not resolve informative missingness. Likewise, the wrapper cannot prevent leakage in a variable already constructed using future records. Fold-specific encoding is necessary but does not establish that an operational prediction design is valid.

Parameter names are standardized where their roles correspond, for example `trees`, `depth`, and `learning_rate`. The wrapper does not pretend that all methods share one tuning scale. Cox `lambda`, MTLR `lambda`, and SVM `cost` enter different objectives. Unknown parameter names are rejected. Resolved settings are retained even when supplied by a default. With

`seed = NULL`, stochastic operations use the caller's random-number generator normally. An explicit integer seed uses **withr**'s scoped helper, restoring the previous RNG state even on error; the package does not manually edit `.Random.seed`. Forest and XGBoost fits without an explicit seed draw and retain an engine seed. Tuning without a seed draws one search seed and records it; checkpoints require an explicit seed so that resumption has a stable identity. Identical seeded resampling plans, rather than repeated unseeded calls, establish matched folds. Temporary engine options, such as forest thread counts, are restored after the operation.

4.3. A common tuning procedure

The tuning signature is

```
surv_tune(formula, data, method = "cox", grid = NULL,
          params = list(), folds = 5L, seed = NULL,
          resamples = NULL, metric = "harrell",
          times = NULL, tau = NULL,
          impute = "none", cache = TRUE, censoring = NULL,
          workers = 1L, checkpoint = NULL, resume = TRUE,
          retry_failed = FALSE, progress = FALSE)
```

A named list is expanded into a Cartesian grid; a data frame can specify selected combinations. With `grid = NULL`, one fixed configuration is evaluated, including an ordinary Cox benchmark. This is resampling of a fixed configuration, not an automatic hyperparameter search; the printed object and `summary()` state this distinction. The same explicit seed and outcome ordering generate the same event-stratified folds across methods. Every candidate uses all folds.

Let $\mathcal{D}_1, \ldots, \mathcal{D}_K$ denote held-out folds and ϑ a candidate setting. The selection rule is

$$\overline{C}(\vartheta) = \frac{1}{K} \sum_{k=1}^{K} \widehat{C}\{\mathcal{D}_k, r_{\widehat{f}_{\vartheta,-k}}\}, \qquad \widehat{\vartheta} = \underset{\vartheta \in \mathcal{G}_{\text{eligible}}}{\arg\max}\, \overline{C}(\vartheta). \tag{36}$$

Encoding and wrapper standardization are estimated using $\mathcal{D} \setminus \mathcal{D}_k$ only. A candidate is eligible only when every fold returns a numerically accepted fit and finite concordance. Errors remain in `fold_results`; they are not replaced by zero or omitted from a favorable mean. Tied eligible means select the first grid row. The winner is refitted using all data supplied to `surv_tune()`, not the external test sample.

The tuning object retains the selected fit in `best_model` and its settings in `best_params`. It also contains `results`, `fold_results`, `grid`, and `fold_id`. Its results can be checked against manual fold-by-fold fitting. Since the maximum in Equation 36 is used for selection, it is not an unbiased assessment of the full search. An independent test sample or nested resampling is required for that purpose. The examples use independent-row stratification. The separate `surv_resamples()` and `surv_validate()` interfaces also support grouped and forward-temporal evaluation of prespecified models. Nested tuning is implemented for stratified outer plans, but explicitly rejected for grouped and temporal plans in this version. These contracts should not be confused with exchangeable random folds.

Selection and reporting criteria are separate. The optional `metric` in `surv_tune()` can select another implemented scalar criterion; error criteria are minimized, whereas discrimination

criteria are maximized. In `surv_validate()`, `metric = "harrell"` explicitly selects inner candidates, independent of the order of reporting `metrics`. For AUC or Brier selection with several reported horizons, `selection_time` identifies one tuning horizon. Required times, truncation and metric names are validated before fitting. For example,

```
R> validation <- surv_validate(form, train, methods = c("cox", "aft"),
+    grids = list(aft = list(distribution = c("weibull", "lognormal"))),
+    metric = "harrell", metrics = c("brier", "harrell"),
+    times = c(90, 180), inner_folds = 3)
```

Reordering `metrics` does not silently change model selection.

For a training formula `form` and data `train`, native neural Cox and optimized additive SVM fits use the same calling pattern:

```
R> nn <- surv_tune(form, train, method = "neural_cox",
+    grid = list(hidden = c(8L, 16L)), folds = 3,
+    params = list(epochs = 100L, ties = "efron",
+                  learning_rate = 0.001, decay = 0.0001))
R> sv <- surv_tune(form, train, method = "svm",
+    grid = list(cost = c(0.01, 0.1)), folds = 3,
+    params = list(kernel = "additive", solver = "newton"))
R> surv_diagnostics(sv)
```

The native neural model requires no Torch, Python, or separate runtime installation. The optimized linear SVM uses base numerical routines; the additive compact eigensystem additionally requires **RSpectra**. The default solver is compact Clarabel, requiring the optional **clarabel** package; there is no silent dense fallback. The historical `quadprog` path remains available explicitly. The additive experiments select `newton`, using base R optimization. Numerical diagnostics distinguish feasibility and optimization certificates from statistical performance. For the neural model, a completed epoch budget is reported as such, not as certified convergence.

4.4. Prediction and scoring

Fit summaries use the usual R generics:

```
R> summary(fitted_model)
R> coef(fitted_model)
R> vcov(fitted_model)
```

The summary reports training size, event count, censoring percentage, parameters, supported predictions, recorded warnings, numerical status, and fitting time. For a tuned object it also reports the selection metric, candidate table and failed folds. Classical coefficients and covariance are exposed only for ordinary Cox and parametric AFT. Original encoded-column labels are retained; AFT coefficients concern log time, whereas Cox summaries also present hazard ratios. Wald intervals condition on the fitted model and are not selection-adjusted intervals. Other learners do not receive fabricated coefficient tables or classical significance tests.

The common interface is not intended to improve the fitted result of an unchanged external engine. On all application training rows, a direct `survival::coxph(form, train, ties = "efron")` and `surv_fit(form, train, "cox")` return identical zero-reference held-out log-risk predictions (maximum absolute difference zero). The additional operations are consistent preprocessing, prediction orientation, optional resampling, and a common inspection contract. This is an interface agreement check, not a runtime comparison with other unified frameworks.

For a fitted or tuned object, the common prediction signature is

```
surv_predict(object, newdata, type = "risk", times = NULL)
```

The `predict()` generic is also available. Risk and median predictions are numeric vectors. Survival predictions are matrices with units in rows and requested horizons in columns. Horizon order and repetition are preserved. New observations require predictors only; outcomes are needed for scoring.

For semiparametric and nonparametric curves, horizons beyond the maximum training follow-up return `NA` with a warning. Parametric AFT curves can extrapolate, and this distinction is explicit. Unsupported prediction types generate an error: a request for SVM probabilities cannot silently invoke an exponential distribution. The score is computed by

```
R> surv_score(fitted_model, test)
```

Training-data scores are resubstitution estimates, not out-of-sample performance.

4.5. Evaluation with conditional censoring weights

The censoring distribution is a nuisance component of several evaluation criteria, not part of every event-model prediction. Define $G(t \mid x) = \Pr(C > t \mid x)$ and $G(t- \mid x) = \Pr(C \geq t \mid x)$. Under conditional independent censoring, adequate positivity, and a transportable reference distribution, inverse probability weights recover contributions missing through censoring (Gerds and Schumacher 2006; Gerds, Kattan, Schumacher, and Yu 2013). A marginal censoring estimator generally does not replace $G(t \mid x)$ when censoring depends on the predictors.

The `surv_censor_fit()` interface estimates a marginal reverse KM, joint-stratum reverse KM, or a Cox working model for censoring. Its formula contains the original failure response `Surv(time, event)`; the caller does not reverse the indicator. For joint categorical strata, separate KM curves are estimated without imposing proportional hazards. For the Cox option, $h_C(t \mid x) = h_{C0}(t) \exp\{z(x)^\top \gamma\}$, with preprocessing learned from the reference data. Conditional independence, correct specification of this working model, and adequate stratum support remain assumptions.

The tie convention deserves attention for recorded daily times. At a time shared by failures and censorings, original failures leave before the censoring hazard is evaluated. Consequently a reverse-KM jump has denominator $n_j - d_j$, not n_j. The Cox implementation represents the same ordering by two ordered phases at each distinct recorded time: original failures depart at the first phase, and censoring events occur at the second. Efron's method handles tied censorings within that second phase. Baseline jumps are mapped back to the original time scale; the event model is not altered and no random time jitter is introduced.

For a native survival prediction $s_i(t) = \widehat{S}(t \mid x_i)$, the conditional-IPCW Brier estimator is

$$\widehat{\mathrm{BS}}(t) = \frac{1}{n} \sum_i \left\{ \frac{\delta_i \mathbb{I}(Y_i \le t) s_i(t)^2}{\widehat{G}(Y_i- \mid x_i)} + \frac{\mathbb{I}(Y_i > t)\{1 - s_i(t)\}^2}{\widehat{G}(t \mid x_i)} \right\}. \tag{37}$$

Censorings at or before t contribute zero but remain in the denominator n. The integrated Brier score uses the normalized trapezoidal integral over the explicitly supplied horizon grid, not an implicit entire lifetime. Let $a_{ij} = \mathbb{I}(r_i > r_j) + \frac{1}{2}\mathbb{I}(r_i = r_j)$ and $w_i^E(t) = \delta_i \mathbb{I}(Y_i \le t)/\widehat{G}(Y_i- \mid x_i)$, $w_j^D(t) = \mathbb{I}(Y_j > t)/\widehat{G}(t \mid x_j)$. The cumulative/dynamic AUC is

$$\widehat{\mathrm{AUC}}(t) = \frac{\sum_{i,j} w_i^E(t) w_j^D(t) a_{ij}}{\sum_{i,j} w_i^E(t) w_j^D(t)}. \tag{38}$$

Both cases and controls receive conditional weights. Sorted control ranks avoid constructing a dense pair matrix.

For concordance truncated at a prespecified τ, the separately named `ipcw_c` criterion uses

$$\begin{aligned} v_{ij}(\tau) &= \frac{\delta_i \mathbb{I}(Y_i \le \tau, Y_i < Y_j)}{\widehat{G}(Y_i- \mid x_i)\widehat{G}(Y_i \mid x_j)}, \\ \widehat{C}_{\mathrm{IPCW}}(\tau) &= \frac{\sum_{i,j} v_{ij}(\tau) a_{ij}}{\sum_{i,j} v_{ij}(\tau)}. \end{aligned} \tag{39}$$

The two factors correspond to observing the earlier failure and retaining the later unit beyond its time. Equal outcome times are excluded and risk ties receive half credit. This criterion is related to conditional-censoring concordance estimation (Gerds *et al.* 2013); its precise discrete tie convention is specified by Equation 39. Historical `uno` retains its marginal weight $1/\widehat{G}(Y_i-)^2$ and is rejected with conditional G. Even with marginal weights these formulas differ at censoring jumps; the names do not conceal that difference. Event-time-group processing avoids quadratic storage but can be slower than the marginal Uno rank-tree routine.

An explicit reference fit is supplied through `censoring_model`:

```
R> g <- surv_censor_fit(form, train, method = "cox")
R> surv_evaluate(fitted_model, test,
+    metrics = c("ipcw_c", "auc", "brier"),
+    times = 90, tau = 180, censoring_model = g)
```

Evaluation data do not replace the training reference. Unsupported horizons, unknown strata, and censoring probabilities below `min_g` are rejected, not silently clipped. A bootstrap resamples assessment cases with the event and censoring fits fixed; it excludes their fitting uncertainty. During tuning and nested validation, `censoring` instead accepts a formula/list specification. The nuisance model is then relearned on each analysis fold, including inner folds, rather than shared from a global fit.

Grouped calibration uses the same explicit censoring fit when supplied. For a fixed prediction group B, the normalized IPCW estimate is

$$\widehat{S}_B(t) = \frac{\sum_{i \in B} w_i^D(t)}{\sum_{i \in B}\{w_i^E(t) + w_i^D(t)\}}. \tag{40}$$

The result is compared with the group's mean predicted survival, not with a risk score. This Hájek-type ratio is not a Kaplan–Meier estimator. It requires the censoring assumptions above and positive weighted mass. Groups are per-horizon midrank bins; tied predictions are not split. The effective sample size is $(\sum b_i)^2 / \sum b_i^2$, with $b_i = w_i^E + w_i^D$. Sparse events, small risk counts and collapsed groups are reported. An optional within-group case bootstrap holds both fitted models and the original groups fixed and gives percentile intervals; undefined repetitions are counted. Without an explicit censoring fit, `surv_calibrate()` retains grouped KM with Greenwood intervals. The `conf_level` argument sets the confidence level in both branches. The IPCW output additionally reports `effective_n` and `bootstrap_valid`; no bootstrap intervals are reported when `n_boot = 0`.

```
R> surv_calibrate(fitted_model, test, times = 90, groups = 4,
+    censoring_model = g, n_boot = 200, seed = 14)
```

Neither estimator recalibrates a model. Its intervals do not include retraining, model selection, censoring-model estimation or full-pipeline uncertainty.

4.6. Interpreting a fixed fitted predictor

Permutation importance and first-order accumulated local effects (ALE) describe the fitted prediction function without changing its estimator. Raw predictor names are recovered from the resolved training terms, including transformations and interactions, rather than from assessment columns or encoded factor contrasts. For a predictor group A, let $\mathcal{D}^{\pi_b(A)}$ permute its original columns jointly across assessment rows. The implemented importance is

$$\widehat{I}_A = \frac{1}{B} \sum_{b=1}^{B} \left[\widehat{C}\{\mathcal{D}, r\} - \widehat{C}\{\mathcal{D}^{\pi_b(A)}, r\} \right]. \tag{41}$$

The same permutations are used across groups and factor contrasts are never permuted independently. Formula transformations are reevaluated after permuting the original columns. Negative importance is retained. Its reported standard deviation describes permutation variability conditional on this model and sample, not a confidence interval or refitting uncertainty. Correlated predictors complicate marginal importance; a final test sample should not be repeatedly used to select variables.

ALE summarizes local prediction changes within observed covariate intervals (Apley and Zhu 2020). For numeric predictor q, let $z_0 < \cdots < z_K$ be empirical quantile boundaries and I_k its bin membership. For target $f(x) = r(x)$ or $f(x) = 1 - \widehat{S}(t_0 \mid x)$, define

$$\begin{aligned} \widehat{d}_k &= \frac{1}{n_k} \sum_{i \in I_k} \{ f(z_k, x_{i,-q}) - f(z_{k-1}, x_{i,-q}) \}, \\ \widehat{A}_k &= \sum_{\ell=1}^{k} \widehat{d}_\ell - \frac{1}{n} \sum_{j=1}^{K} n_j \sum_{\ell=1}^{j} \widehat{d}_\ell. \end{aligned} \tag{42}$$

The returned values are evaluated at upper bin boundaries and centered by empirical bin frequencies. Repeated boundaries and empty bins are merged. This is an explicit binwise first-order approximation, without smoothing, confidence intervals, or a causal interpretation. Failure-probability ALE requires native supported survival curves; risk ALE retains the model-specific score scale. The public calls are

```
R> importance <- surv_importance(fitted_model, test,
+    n_repeats = 5, seed = 20261024)
R> effects <- surv_ale(fitted_model, test,
+    variable = "temperature", bins = 10,
+    type = "failure", time = 180)
```

Base-graphics methods are supplied for both returned objects.

4.7. Reliability summaries at an attained time

For fixed covariates and attained model time a with $S(a \mid x) > 0$, the conditional survival of the remaining duration is

$$S_a(u \mid x) = \Pr(T - a > u \mid T > a, x) = \frac{S(a + u \mid x)}{S(a \mid x)}, \qquad u \geq 0. \tag{43}$$

The interface returns $F_a(u \mid x) = 1 - S_a(u \mid x)$ and

$$\mu_{a,\tau}(x) = \int_0^\tau S_a(u \mid x)\, du, \qquad q_{a,p}(x) = \inf\{u \in [0, \tau] : S_a(u \mid x) \leq 1 - p\}. \tag{44}$$

The restricted mean uses trapezoidal integration on zero, the requested horizons, and an equally spaced augmented grid. Quantiles are first grid crossings, not interpolated exact failure times. An empty crossing set returns `NA` and an explicit status. Neither the restricted mean nor an unreached median is an estimate of an unrestricted life expectancy or cure.

The `age` argument refers to the model's time origin; for the present cohort this is time since first recorded observation, not installation age. Equation 43 conditions on survival while keeping covariates fixed. It is not a longitudinal updating model. By default, $a+\tau$ cannot exceed training follow-up. Only supported parametric engines allow an explicit `extrapolate = TRUE`; this adds a tail assumption, not observed information. Risk-only methods are rejected. For example,

```
R> remaining <- surv_reliability(fitted_model, new_drives,
+    age = 30, horizons = c(60, 150), probs = c(0.01, 0.5),
+    grid_points = 501)
R> remaining$summary
R> remaining$quantiles
```

4.8. Local parallel tuning and checkpoint recovery

Default tuning remains serial and disk-free. Setting `workers > 1` uses local PSOCK processes from **parallel**. Work is grouped by analysis fold, so all compatible SVM candidates in that fold retain preprocessing, Hessian, and solver-workspace reuse. Each fit uses the same recorded seed regardless of scheduling; candidate-major result ordering and the caller's random state are preserved. Engine threads must be one when explicitly specified with multiple workers. Process startup, serialization, and duplicated workspaces mean that speedup is not guaranteed and memory use can increase.

A supplied `checkpoint` directory stores individually committed candidate-fold records and, on completion, the winning refit. Compatibility checks include the complete data and row

order, formulas, resampling plan, parameters, seed, RNG/contrast settings, and recorded software versions. Changed predictors cannot silently reuse scores. Failed or undefined folds remain ineligible; `retry_failed = TRUE` explicitly retries them. Portable formulas support standard functions and data columns, not arbitrary captured closures. Missing work rebuilds numerical workspaces: checkpoint recovery does not serialize a solver's warm-start history.

Each directory has one live owner, with a token checked before record commits. Ordinary errors and interrupts release its lock. A hard process crash can leave a stale lock; the user must verify that no run remains before removing it. This is local recovery, not a distributed fault-tolerance guarantee. Corrupt or incompatible records produce errors rather than candidate wins.

```
R> tuned <- surv_tune(form, train, "svm",
+    grid = list(cost = c(0.01, 0.1)), folds = 3,
+    params = list(kernel = "linear", solver = "clarabel"),
+    workers = 2, checkpoint = "owned-search", progress = TRUE)
R> tuned$execution
```

Repeating the same call reuses completed records and a compatible completed winning fit. The number of computed and reused folds remains inspectable.

5. A controlled example with all methods

5.1. Data generation and matched fitting

We consider one generated reliability data set to illustrate the complete interface. It is not a repeated simulation study. There are 1,560 independent units with independent covariates: temperature $X_1 \sim N(30, 5^2)$, power-on hours $X_2 \sim U(1000, 30000)$, load $X_3 \sim U(0, 1)$, and binary design X_4 with equal probabilities. Let $Z_1 = (X_1 - 30)/5$ and $Z_2 = (X_2 - 15500)/14500$. Failure and censoring times are generated by

$$\log T = \log 90 - 0.5Z_1 - 0.25Z_2 - 0.45X_3^2 + 0.3Z_1X_3 + 0.25\mathbb{I}(X_4 = B) + 0.7\varepsilon, \quad \varepsilon \sim N(0, 1), \tag{45}$$

$$C = \min(C^*, 365), \qquad C^* \sim \text{Exp}(1/160). \tag{46}$$

The censoring variable is independent of covariates and failure time. The first 360 units form training and the remaining 1,200 form test. With seed 20261020, the samples contain 211 and 650 events, respectively. Training censoring is 41.4%. All 23 methods use the same training units, the same four covariates, and three-fold tuning with seed 20261021. The fitted formula does not explicitly include the squared load or interaction in Equation 45.

Small grids keep replication accessible. Gehan penalties use $\lambda \in \{0.01, 0.1\}$, Cox penalties use $\lambda \in \{0.001, 0.01\}$, and elastic-net candidates use $\alpha \in \{0.25, 0.75\}$. The parametric AFT candidates are Weibull, log-normal, and log-logistic. The tree uses complexity parameters 0.001, 0.01, or 0.05. The forest uses 200 trees and node sizes 15 or 50. Boosting considers 100 or 200 stages; XGBoost Cox also considers depths one or two, while XGBoost AFT considers

normal-error scales 0.5 or 1. MTLR uses five or ten intervals and $\lambda \in \{0.1, 1\}$. The SVM grid combines costs 0.01 or 0.1 with linear or additive kernels. Both use the compact Newton solver. The spline Cox degrees of freedom are two or three, flexible parametric interior-knot counts are zero or one, and componentwise CoxBoost uses 50 or 100 steps. The alternative forests use 200 trees, with node sizes 15 or 50 for **ranger** and 5 or 15 for **aorsf**. Native neural Cox considers 8 or 16 hidden units, 100 epochs, learning rate 0.001, weight decay 0.0001, and Efron ties. The discrete neural model uses 4 or 8 hidden units, eight intervals, weight decay 0.01, and at most 1,000 optimization iterations. Kaplan–Meier has no tuning parameter. All fixed and selected settings are written to `selected_settings.csv`.

5.2. Fitting and interpreting the obtained results

After creating `sim_train` and `sim_test` with the replication script, elastic-net Cox estimation is performed as follows:

```
R> f <- Surv(time, event) ~ temperature + power_hours + load + design
R> tuned <- surv_tune(f, sim_train, method = "cox_elasticnet",
+    grid = list(lambda = c(0.001, 0.01), alpha = c(0.25, 0.75)),
+    folds = 3, seed = 20261021)
R> tuned$best_params
R> surv_score(tuned, sim_test)
R> surv_predict(tuned, sim_test[1:2, ], type = "survival",
+    times = c(30, 90))
```

The calling sequence is retained for other methods by changing the identifier and appropriate grid. A median request can be made for `aft` or `xgboost_aft`; it is intentionally unavailable for `cox_elasticnet` in this version.

From Table 3, we observe that the Gehan AFT and Cox configurations produce similar test concordance, approximately 0.685–0.686. The MTLR result is 0.687, and the single tree returns 0.630. Differences between several regression configurations are small relative to their conditional standard errors. These observations concern one training sample, one test sample, and the specified grids; they do not establish a preferred family under other nonlinearities, censoring patterns, or sample sizes.

The optimized SVM gives test concordance 0.685, and native neural Cox gives 0.665. The latter is not uniformly superior to the linear models: the generated mechanism in Equation 45 is a log-normal AFT mechanism, and the neural Cox model still imposes proportional hazards. A flexible covariate map does not remove that restriction. There are 0 failed candidate-fold fits under the recorded settings. The replication output retains warnings separately from errors; an accepted fit need not be free of engine warnings.

6. A historical reliability application

6.1. Cohort construction and descriptive analysis

We consider the public Backblaze Drive Stats data for 2013 (Backblaze 2026). The source comprises 266 daily snapshots from April 10 to December 31, containing 5,091,501 records.

Method	CV C	Test C	SE	Seconds
Parametric AFT	0.687	0.686	0.011	0.1
Gehan AFT LASSO	0.689	0.685	0.011	2.6
Gehan AFT ridge	0.688	0.686	0.011	2.0
Gehan AFT elastic net	0.689	0.685	0.011	2.9
Cox	0.686	0.685	0.011	0.0
Cox LASSO	0.685	0.685	0.011	0.2
Cox ridge	0.686	0.685	0.011	0.1
Cox elastic net	0.686	0.685	0.011	0.2
Survival tree	0.600	0.630	0.012	0.1
Random survival forest	0.663	0.679	0.011	0.7
Cox gradient boosting	0.633	0.678	0.011	0.2
XGBoost Cox	0.634	0.680	0.011	0.3
XGBoost AFT	0.638	0.677	0.011	0.6
MTLR	0.688	0.687	0.011	0.5
Censored SVM	0.678	0.685	0.011	1.0
Kaplan–Meier	0.500	0.500	0.000	0.0
Spline Cox	0.661	0.683	0.011	0.1
Flexible parametric	0.685	0.685	0.011	0.8
Componentwise CoxBoost	0.686	0.685	0.011	1.7
Ranger forest	0.634	0.673	0.011	1.7
Oblique forest	0.668	0.682	0.011	0.5
Native neural Cox	0.659	0.665	0.012	0.3
Discrete neural survival	0.639	0.659	0.012	0.4

Table 3: Controlled generated example: mean tuning concordance, independent test concordance, conditional infinitesimal-jackknife standard error, and elapsed seconds for tuning, the winning refit, held-out prediction, and concordance calculation. These one-machine timings are not a controlled scalability benchmark.

The processed cohort has one row per drive serial number. If d_{i0} is the first recorded date and d_{i1} is the failure or final recorded date, the observed duration is $Y_i = d_{i1} - d_{i0}$ elapsed days. The event indicator identifies a recorded failure; otherwise the drive is treated as right-censored. Predictions are made after the first snapshot, conditional on being operational at that snapshot.

The source contains 29,072 drives and 724 recorded failures. Six drives already fail at their first snapshot, and eight other drives have no positive post-entry follow-up. These 14 histories are retained in the supplied source file with explicit exclusion reasons but are not members of the subsequent failure risk set. The analytic cohort has 29,058 drives, 718 subsequent failures and 28,340 censored observations (97.53% censoring). Both the median and maximum observed duration are 265 days. These exclusions define the prediction target; they are not computational subsampling for an SVM. The median observed duration is not the median uncensored failure time. Table 4 describes the new split used for this software article. These are new package fits, not performance values copied from the earlier chapter.

Figure 1 presents the Kaplan–Meier estimates by capacity. The curves describe relatively small failure probabilities over the observed window. Capacity should not be interpreted as an isolated physical cause: manufacturer, model, operating history, and entry date can differ between groups. Independent censoring is an assumption, not a property verified by the public

Sample	Drives	Failures	Censored	Censored (%)
Cohort	29,058	718	28,340	97.53
Training	23,246	574	22,672	97.53
Test	5,812	144	5,668	97.52

Table 4: Backblaze cohort and event-stratified random 80/20 split. Durations begin at first recorded observation, not at installation.

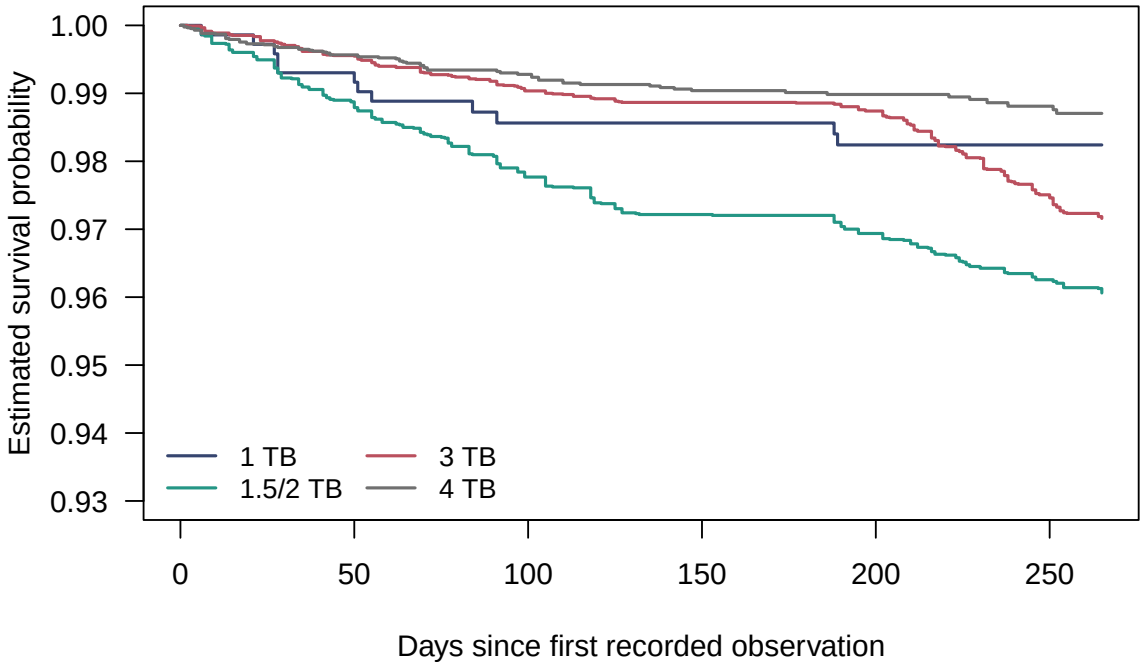


Figure 1: Kaplan–Meier estimates by drive capacity in the complete 2013 cohort. The vertical scale is restricted to make the differences visible. The 1.5/2 TB group pools both nominal capacities. Capacity groups also differ in other characteristics and calendar entry.

failure indicator. The original encoding rounds capacity in decimal terabytes. Consequently, the level stored as `2 TB` contains 3,102 nominal 1.5 TB drives with 264 failures and 5,198 nominal 2 TB drives with 55 failures. We display this level as 1.5/2 TB and retain its original encoding in all fits. The supplied byte-level table makes the pooling explicit; these counts do not isolate a capacity effect.

The seven predictors are defined in Table 5. Diagnostic variables come from self-monitoring, analysis, and reporting technology (SMART) attributes. The transformation $\log(1+x)$ retains zeros and compresses a right-skewed count or operating-time scale. It does not make the variable normally distributed, and predictor normality is not required by these models. In this cohort, median power-on hours are 7,469 (interquartile range 869–17,250), and median recorded temperature is 26 (interquartile range 23–29). Both sector-count variables have zero at their first and third quartiles, with large positive maxima. Manufacturer and model-specific interpretation of raw SMART values remains relevant.

The reconstruction script reads the 266 original daily files in date order and fixes all four

Predictor	Definition
Manufacturer	Factor derived from recorded model.
Capacity group	Factor: 1, pooled 1.5/2, 3, or 4 TB.
`log_power_hours`	$\log(1 + \text{SMART}_9)$ at the first snapshot.
`temperature`	Raw SMART 194 at the first snapshot.
`log_reallocated`	$\log(1 + \text{SMART}_5)$ at the first snapshot.
`pending`	Indicator that entry-snapshot SMART 197 is positive.
`entry_day`	Days between first drive observation and April 10, 2013.

Table 5: Predictor definitions. Selected SMART measurements are taken from the exact first snapshot, never from a later nonmissing value. The retained baseline date permits a direct availability audit.

SMART measurements at each drive's first row. It never updates a baseline value from later records. All four selected attributes are already observed at entry for every source drive; this is verified rather than inferred from a completed aggregate. The baseline date, eligibility, source counts and missingness audit are exported. No imputation is performed. This resolves baseline-information timing for the stated landmark, but does not turn a historical random split into prospective validation.

The time origin also limits interpretation. Drives already operating at the start of the window are not new components. The estimand concerns follow-up from first appearance, conditional on cohort inclusion, not lifetime from installation. The data are not represented as a recurrent-failure or repairable-system counting process.

6.2. Training, tuning, and held-out comparison

The 80/20 split is made by drive, stratified by event, with seed 20261022. The training sample has 23,246 drives and 574 failures; the test sample has 5,812 drives and 144 failures. Test rows do not enter selection. All candidates use the same three training folds with seed 20261023. The split is random, not chronological, and does not assess transfer to a later year or an unseen model. Grids are those in Section 5. For example, the forest is fitted by

```
R> form <- Surv(time_days, event) ~ manufacturer + capacity_group +
+    log_power_hours + temperature + log_reallocated +
+    pending + entry_day
R> forest <- surv_tune(form, train, method = "rsf",
+    grid = list(min_node = c(15L, 50L)),
+    params = list(trees = 200L, threads = 1L),
+    folds = 3, seed = 20261023)
R> surv_score(forest, test)

[1] 0.8558433
```

The application fits 14 configurations on the entire training sample, including native neural Cox and separate linear and additive SVM searches. The linear kernel uses Clarabel and the additive kernel uses Newton, both with costs 0.01 and 0.1 and the same folds as the other methods. No event-enriched subset is inserted into this comparison. The three Gehan AFT

Method	CV C	Test C	SE	Seconds
Parametric AFT	0.795	0.823	0.017	2.2
Cox	0.795	0.824	0.017	0.6
Cox LASSO	0.788	0.802	0.016	1.7
Cox ridge	0.796	0.825	0.018	13.3
Cox elastic net	0.788	0.800	0.016	10.1
Survival tree	0.792	0.814	0.016	1.0
Random survival forest	0.846	0.856	0.015	99.6
Cox gradient boosting	0.812	0.829	0.018	7.4
XGBoost Cox	0.834	0.837	0.019	4.5
XGBoost AFT	0.824	0.823	0.017	10.2
MTLR	0.792	0.813	0.017	17.1
SVM (linear)	0.774	0.799	0.020	22.7
SVM (additive)	0.700	0.734	0.022	215.2
Native neural Cox	0.795	0.813	0.017	8.4

Table 6: New Backblaze fits from the package. CV is the largest eligible three-fold mean concordance. Test SE is the conditional infinitesimal-jackknife standard error. Seconds include the complete small-grid search, winning refit, held-out prediction, and concordance calculation.

configurations are exercised on the common generated sample only; their omission from the large-data comparison is a computational scope choice, not evidence of inferior prediction. Other recently added reference adapters are also illustrated in the controlled example rather than treated as a full large-data benchmark.

From Table 6, we observe that RSF has the largest mean tuning concordance (0.846) and test concordance (0.856) among the fitted configurations. XGBoost Cox gives test concordance 0.837, followed by Cox gradient boosting (0.829). The ordinary Cox and selected Weibull AFT fits give approximately 0.824 and 0.823, respectively. These results support discrimination within this historical split. They do not establish that the difference between RSF and XGBoost Cox is significant: the pointwise intervals in Figure 2 are not a paired test.

Native neural Cox gives test concordance 0.813, compared with 0.799 for linear SVM and 0.734 for additive SVM. The kernel with greater flexibility does not necessarily give greater held-out concordance. These comparisons use the common training folds, whereas the fixed-budget engineering experiments in Section 8 examine numerical and computational properties.

Table 7 records the selected parameters. There are 6 failed candidate-fold fits in the real-data search. The log-normal and log-logistic AFT candidates give nonfinite risk predictions in two and three folds, respectively, and are excluded. The smaller Cox LASSO penalty fails in one fold with an overflow/underflow error. The selected Weibull AFT and larger Cox LASSO penalty succeed in every fold. All candidate errors remain in the replication outputs. The standard errors describe held-out concordance variation conditional on fitted scores, not variation from repeating the split, selecting parameters again, or changing the population of drives.

The ordinary Cox search also produces possible infinite-coefficient warnings for sparse factor categories in training folds. Its finite risk predictions remain available, but the warning is retained in `engine_warnings.txt`; the corresponding coefficients are not interpreted as stable hazard-ratio estimates. These warnings differ from failed folds: the selection rule excludes

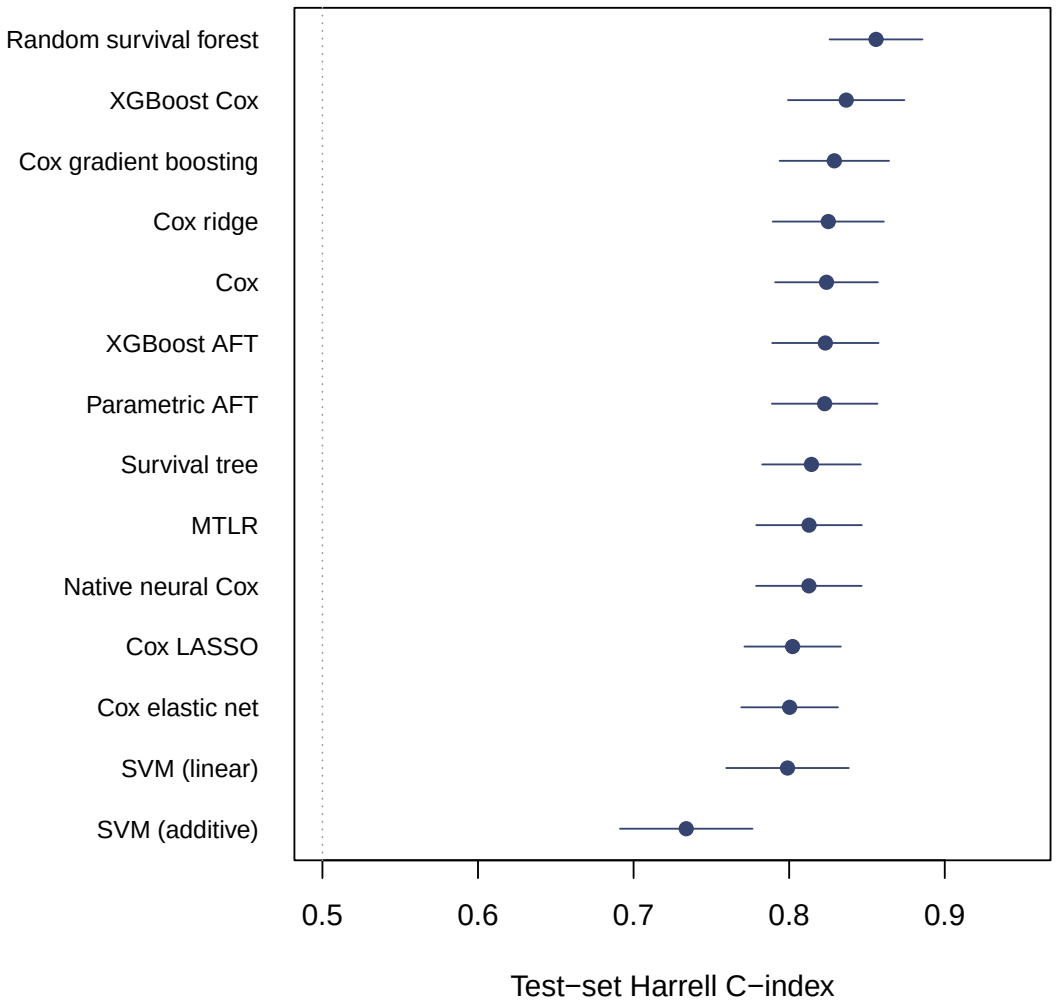


Figure 2: Held-out concordance in the new Backblaze application. Segments are pointwise normal intervals conditional on the fitted predictors. They exclude split and tuning variability and are not simultaneous pairwise comparison intervals.

errors, failed numerical certificates, and nonfinite concordance, not every warning returned by an engine.

The tables distinguish search time from prediction quality. Search grids have different sizes and meanings, so elapsed times are not equal-budget algorithmic comparisons. A scalability study would require repeated timings, controlled hardware, memory measurements, and several training sizes. The present search timings document replication cost only. The separate fixed-configuration and fresh-process measurements in Section 8 address computational behavior more directly.

6.3. Prediction for additional covariate profiles

We consider three hypothetical 3 TB Hitachi profiles entering on the first day of the 2013 window. Power-on hours are 5,000, 15,000, and 30,000; temperatures are 25, 30, and 35;

Method	Selected parameters
Parametric AFT	Weibull distribution
Cox	No tuning parameter
Cox LASSO; ridge	$\lambda = 0.01$ in both cases
Cox elastic net	$\lambda = 0.01$, $\alpha = 0.75$
Tree	Complexity 0.001; node size 10; maximum depth 5
RSF	200 trees; node size 50; `mtry = 3`; five split points
Cox gradient boosting	200 stages; depth 2; rate 0.03; bag fraction 0.7
XGBoost Cox	200 trees; depth 2; rate 0.05
XGBoost AFT	200 trees; depth 2; rate 0.05; normal error; scale 1
MTLR	Ten intervals; $\lambda = 0.1$
Native neural Cox	16 hidden units; 100 epochs; Efron ties
Linear SVM	Cost 0.1; compact Clarabel solver
Additive SVM	Cost 0.01; compact Newton solver

Table 7: Selected Backblaze configurations. XGBoost leaf penalty and minimum child weight are both 1; Cox gradient boosting minimum node size is 10. Forest and XGBoost use one thread. Complete settings are exported by the script.

reallocated-sector counts are 0, 10, and 100; pending-sector indicators are 0, 0, and 1, respectively. These are constructed profiles, not additional observed failures. Several predictors change together, so their contrasts are not isolated covariate effects. Factor levels are those learned from training:

```
R> S <- surv_predict(forest, new_drives, type = "survival",
+    times = c(90, 180))
R> round(1 - S, 4)

       t=90  t=180
[1,] 0.0008 0.0011
[2,] 0.0097 0.0163
[3,] 0.1148 0.1387
```

The matrix contains modeled failure probabilities. For example, profile 3 has a modeled failure probability of approximately 13.87% by 180 days under this fitted forest. A call with `type = "risk"` instead returns mortality scores; these must not be substituted for the matrix probabilities.

Figure 3 presents the RSF curves, and Table 8 gives horizon-specific probabilities from five methods. Differences between their probabilities are not resolved by a common discrimination criterion. Calibration and the timing of covariate availability require separate assessment. A parametric median beyond 265 days would depend on extrapolation and is not interpreted as observed remaining life.

For another interpretation of the forest score, three training-score quartiles define four groups. The cutpoints are applied unchanged to test scores, avoiding thresholds selected from test outcomes. Figure 4 presents test-sample Kaplan–Meier curves. The lowest group has four failures among 1,501 drives, whereas the highest has 117 among 1,421. These crude counts and percentages are distinct from Kaplan–Meier failure probabilities. Separation indicates

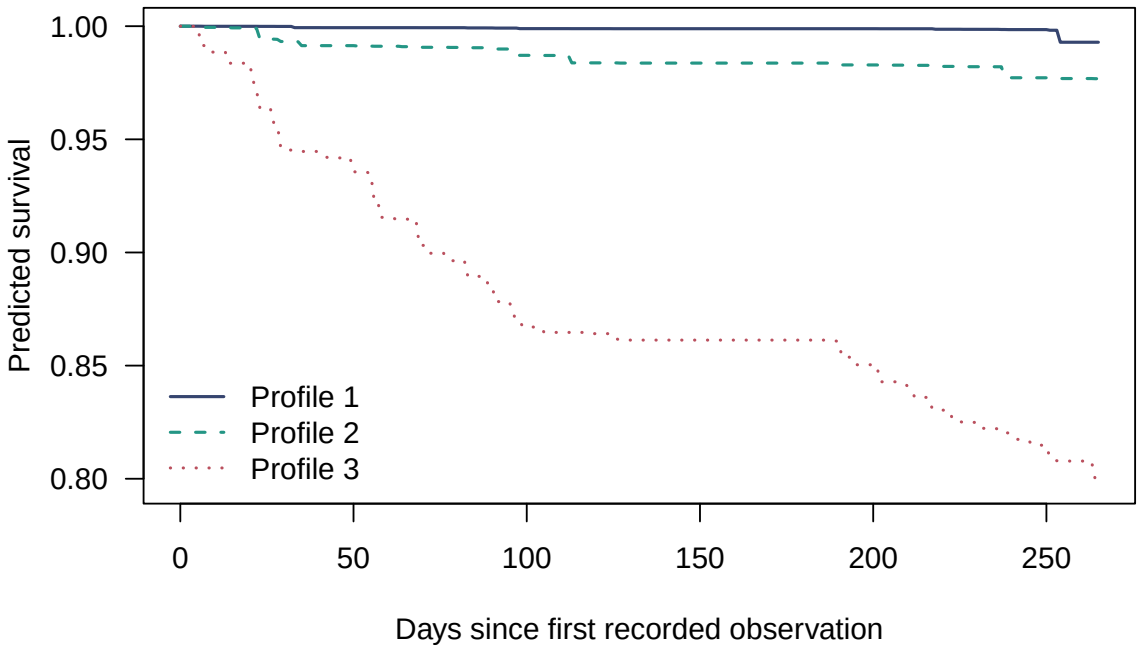


Figure 3: Fitted RSF survival curves for three constructed profiles. These are outputs of a historical model, not externally validated predictions for current hardware or Backblaze operations.

risk-ranking information, not a validated maintenance policy. Replacement costs, failure consequences, calibration, and prospective performance would also be required for that decision.

Method	Profile	$\widehat{F}(90)$	$\widehat{F}(180)$
Random survival forest	1	0.0008	0.0011
Random survival forest	2	0.0097	0.0163
Random survival forest	3	0.1148	0.1387
XGBoost Cox	1	0.0017	0.0024
XGBoost Cox	2	0.0025	0.0037
XGBoost Cox	3	0.1209	0.1698
XGBoost AFT	1	0.0002	0.0024
XGBoost AFT	2	0.0003	0.0028
XGBoost AFT	3	0.1247	0.3232
MTLR	1	0.0066	0.0107
MTLR	2	0.0141	0.0225
MTLR	3	0.0945	0.1366
Native neural Cox	1	0.0065	0.0093
Native neural Cox	2	0.0124	0.0175
Native neural Cox	3	0.0441	0.0622

Table 8: Modeled failure probabilities for the constructed profiles under five adapters returning survival curves. Concordance does not establish that these probabilities are calibrated.

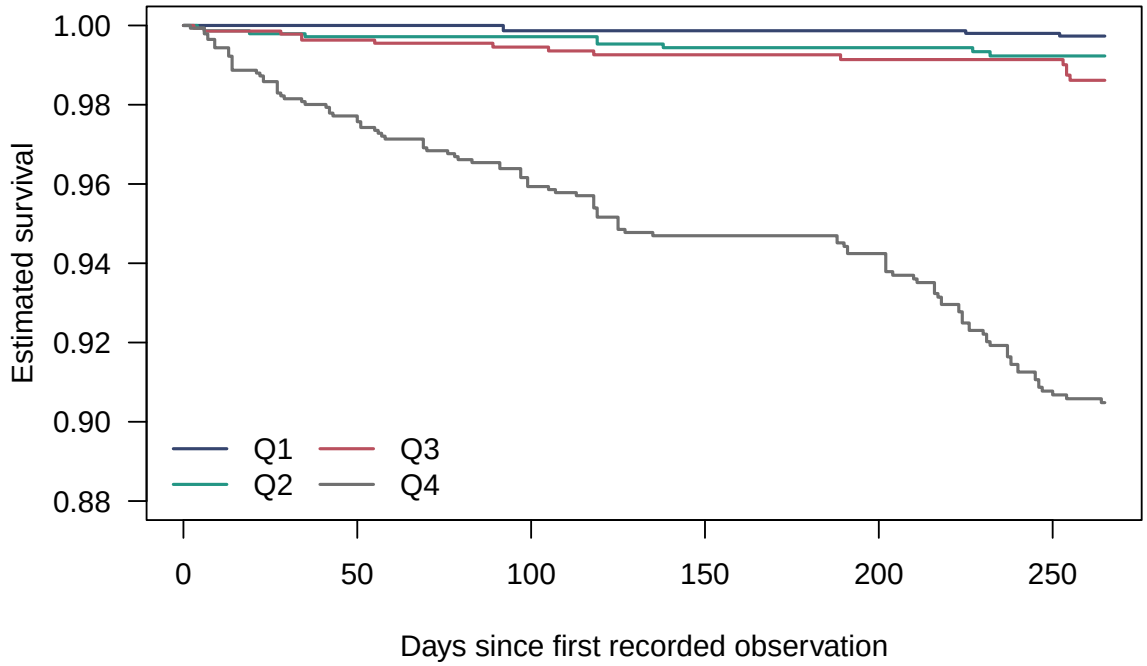


Figure 4: Test-sample Kaplan–Meier curves for RSF score groups, defined using training-score quartiles. Q4 is the highest-score group. This is descriptive risk stratification, not probability calibration. The vertical scale is restricted to display the differences over the observed window.

6.4. Probability assessment and conditional calibration

Harrell concordance remains the tuning criterion. We separately assess native probabilities against a covariate-free KM reference. The administrative archive ends on December 31, so a drive entering near that date cannot support a 180-day assessment. The prespecified probability target therefore contains drives entering by July 4, 2013. This gives 19084 reference training drives and 4751 held-out drives, with 137 recorded test failures. Event models remain fitted to all 23,246 eligible training drives. Only the assessment population and its censoring reference change.

A reverse-KM censoring model uses prespecified 30-day calendar-entry bands. This avoids imposing proportional hazards on administrative entry patterns. Nonfailure withdrawal must still be conditionally independent within the bands, and the reference must transport to the assessment data. This assumption is not established by a positive weight diagnostic. Brier scores use Equation 37; IBS is the normalized trapezoidal integral over 30, 60, 90, 120, 150 and 180 days. No test-driven retuning or censoring-weight clipping is performed.

Method	Brier(90)	Brier(180)	IBS(30–180)
Kaplan–Meier	0.01125	0.01726	0.01312
Parametric AFT	0.01019	0.01534	0.01193
Cox	0.01013	0.01544	0.01191
Cox LASSO	0.01052	0.01617	0.01238
Cox ridge	0.01044	0.01596	0.01227
Cox elastic net	0.01050	0.01613	0.01236
Random survival forest	0.01004	0.01537	0.01180
XGBoost Cox	0.01002	0.01547	0.01185
XGBoost AFT	0.01025	0.01523	0.01193
MTLR	0.01043	0.01596	0.01225
Native neural Cox	0.01087	0.01655	0.01266

Table 9: Native probability error in the prespecified early-entry target. Lower values are preferred. All rows use the same reference censoring fit; risk-only learners are not assigned invented probability curves.

From Table 9, we observe an IBS of 0.01312 for KM and 0.01180 for RSF. XGBoost Cox gives 0.01185, whereas XGBoost AFT has the smallest 180-day Brier score, 0.01523. Thus the ranking depends on the prediction target and criterion. Small absolute errors partly reflect the rarity of failures; comparison with KM is essential. These point estimates are not a paired significance test or evidence of prospective calibration.

Figure 5 uses the normalized IPCW ratio in Equation 40, not a marginal within-group KM. At 180 days, the lowest-survival group has mean predicted survival 0.9512 and weighted observed survival 0.9434 (interval 0.9278–0.9567). The highest-survival group has no failures by either horizon, and another 90-day group has only two failures. Their `low_support` flags remain in the exported table. The fixed-fit bootstrap cannot infer an unobserved rare-event tail or account for model fitting. Calibration is therefore presented as a qualified diagnostic, not as confirmation of a maintenance decision rule.

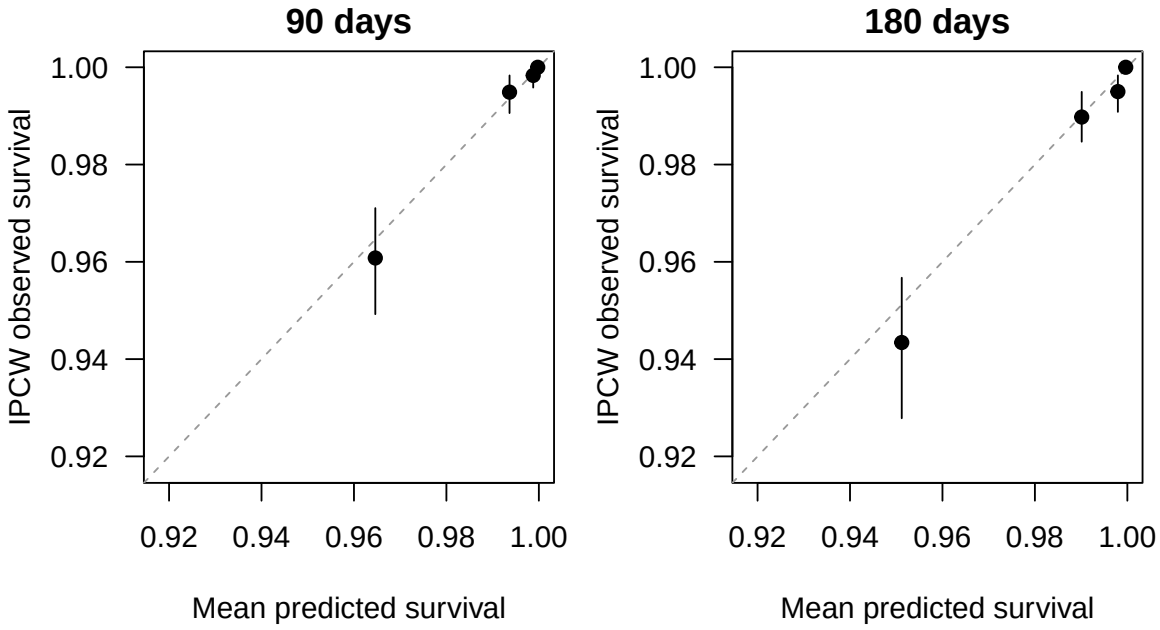


Figure 5: RSF grouped calibration at 90 and 180 days in the early-entry test target. Groups use predicted-survival midranks. Vertical intervals are 95% percentiles from 200 within-group bootstrap repetitions with the fitted event model, censoring model and groups fixed. The diagonal denotes agreement. The high-survival groups include sparse or zero events; degenerate bootstrap intervals at one do not imply certainty.

7. Extended workflow examples

7.1. Held-out interpretation and remaining duration

We now apply the post-fit interfaces to the same Backblaze models and all 5,812 test drives, without selecting variables or refitting from this sample. Table 10 reports Equation 41 for Cox, RSF, and native neural Cox, using five common permutations and seed 20261024. The table distinguishes variation over permutations from model uncertainty. The effects depend on the fitted function and available predictors; a larger importance is not a larger physical or causal effect. From Table 10, we observe that manufacturer has the largest mean importance for all three fits. In RSF, its decrease of 0.144 is followed by power-on hours (0.078) and capacity (0.074). Native neural Cox assigns slightly negative importance to temperature: permuting this input improves concordance in this assessment. This is not evidence of a protective physical effect; it describes a limitation of the fitted ranking in this sample.

Figure 6 combines RSF importance with first-order ALE for recorded SMART temperature on $F(180 \mid x)$. Quantile bins use the assessment distribution; the plotted effects are centered upper-bin values from Equation 42. SMART encodings can vary by manufacturer, so the horizontal scale is not asserted to be a universal physical temperature scale. An ALE curve describes this fitted predictor, not an intervention on drive temperature. These effects do not establish transfer to a later hardware fleet.

For the three additional profiles in Table 8, we condition on survival to model day $a = 30$. Table 11 reports failure probabilities over the next 60 and 150 days, the restricted mean over 150 days, and the 1% remaining-time quantile when reached. The augmented grid has a maximum spacing of 0.3 day. A reported value is therefore a grid crossing, not an exact predicted failure date. Medians that are not reached remain missing in the exported quantile table. These conditional summaries use fixed profile covariates and do not constitute prospective sensor updates. For profile 3, the probability of failure over the next 150 days is 0.0888, conditional on reaching day 30, and its restricted remaining mean is 140.65 days out of that 150-day horizon. Its 1% quantile is reached after 20.1 additional days; profiles 1 and 2 do not reach that probability within the horizon. None reaches the median. These small tail probabilities explain why an apparently precise unrestricted "remaining life" number would not be supported by these curves.

7.2. A controlled conditional-censoring example

To illustrate the nuisance-model contract separately from the retrospective cohort, we generate independent training and test samples of size 2,000 each, using seed 20261025. Let $X_1 \sim \mathcal{U}(-1, 1)$ and $X_2 \sim \text{Bernoulli}(1/2)$ independently, and generate

$$\begin{aligned} T \mid x &\sim \text{Exp}\{0.02 \exp(0.8x_1 - 0.3x_2)\}, \\ C^* \mid x &\sim \text{Exp}\{0.03 \exp(x_1 + 0.2x_2)\}, \qquad C = \min(C^*, 40), \end{aligned} \tag{47}$$

with independent exponential draws given x. Rates, rather than mean parameters, are shown. The training censoring percentage is 73.0%. Before the administrative endpoint, the Cox censoring specification is correct; the evaluation horizons 10 and 15 are strictly earlier than that endpoint.

Predictor	Cox	RSF	Neural Cox
manufacturer	0.1905 (0.0256)	0.1442 (0.0144)	0.1361 (0.0204)
capacity_group	0.0325 (0.0066)	0.0745 (0.0085)	0.0231 (0.0084)
log_power_hours	0.0325 (0.0063)	0.0776 (0.0166)	0.0070 (0.0047)
temperature	0.0010 (0.0022)	0.0130 (0.0072)	-0.0048 (0.0043)
log_reallocated	0.0383 (0.0034)	0.0464 (0.0040)	0.0485 (0.0048)
pending	0.0133 (0.0018)	0.0075 (0.0020)	0.0095 (0.0029)
entry_day	0.0056 (0.0087)	0.0274 (0.0074)	0.0000 (0.0060)

Table 10: Decrease in held-out Harrell concordance after permuting each original predictor. Parentheses contain permutation SD, not confidence intervals. All test rows are retained.

Profile	$F_a(60)$	$F_a(150)$	$\mu_{a,150}$	$q_{a,.01}$
1	0.0007	0.0010	149.88	Not reached
2	0.0030	0.0096	149.11	Not reached
3	0.0636	0.0888	140.65	20.1

Table 11: RSF summaries conditional on survival to day 30 since first recording. Durations and restricted means are in days. A quantile marked "Not reached" has no crossing within the largest future horizon of 150 days.

An event Cox model is fitted using only the observed training data. We then evaluate this same predictor three ways: using complete generated test failure times, marginal training reverse KM, and conditional training Cox G. The complete-data column is available only because failure times were generated; it is not obtainable by uncensoring a real reliability dataset.

Table 12 illustrates that the censoring model is part of the evaluation specification. Harrell concordance is unchanged between the two observed-outcome columns because it does not use G. Weighted criteria can differ because case and control contributions differ. One realization does not establish that conditional weighting always reduces finite-sample error; misspecification and small censoring probabilities can also increase error or instability. The nuisance model uses training rows only. Separate fold-level tests verify that tuning relearns it within each analysis subset. Here conditional AUC is 0.6763, compared with 0.6923 under marginal weights and 0.6573 using complete outcomes. Conversely, conditional Brier is 0.1439, whereas marginal Brier is 0.1369 and the complete-outcome value is 0.1384. Conditional weighting is therefore not uniformly closer to the complete-data reference in this realization. Its purpose is to use the appropriate observation-probability model under the stated design, not to guarantee an improvement in every finite sample.

7.3. Full-training parallel and resume comparison

We use every one of the 23,246 training drives to compare serial and two-worker PSOCK tuning of linear SVM with Clarabel, the same two costs, three folds, and seed 20261023. Both fresh runs write checkpoints in separate new owned directories. Wall time includes process startup where applicable, checkpoint I/O, all candidate fits, and the winning refit; external test prediction is outside the timing. The third call repeats the completed parallel search.

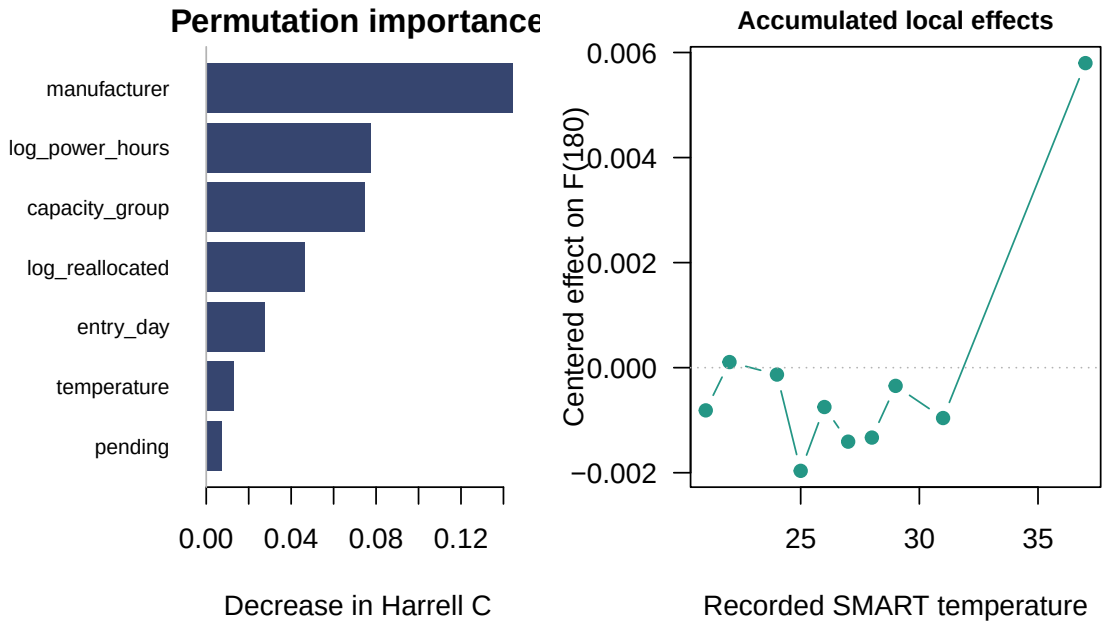


Figure 6: RSF interpretation on every held-out drive. Left: mean permutation importance. Right: centered binwise local effects on 180-day failure probability. No inferential uncertainty band or causal interpretation is added.

Criterion	Complete outcomes	Marginal G	Conditional G
Harrell C	0.6230	0.6340	0.6340
Pair-weighted $C(15)$	0.6428	0.6438	0.6488
AUC(10)	0.6573	0.6923	0.6763
Brier(10)	0.1384	0.1369	0.1439

Table 12: One controlled realization under covariate-dependent censoring. Every column evaluates the same event-model predictions. Complete outcomes provide a finite-sample reference, not a population value.

The maximum absolute difference between serial and parallel fold concordances is 0, and between their held-out risk predictions is 0. Both select the same grid row. In Table 13, serial execution takes 23.45 seconds and the two-worker run takes 50.86 seconds, approximately 2.17 times as long. Thus parallel execution is slower for this measured workload; two workers do not guarantee a speedup. Completed-search reuse takes 2.81 seconds and avoids all six fold fits and the winning refit. A separate regression test removes one committed fold and the winning-refit fixture, then verifies recovery with five reused folds and one newly computed fold. No observation reduction or kernel substitution is used. Table 13 reports elapsed time only: aggregate memory across the parent and workers was not measured in this experiment. The separate process-memory audits below concern the stated SVM and neural configurations, not this PSOCK run. Other models or cheaper searches may be slower with multiple workers.

Run	Seconds	New fits	Reused folds	Refit reused
Serial	23.45	6	0	No
Two workers	50.86	6	0	No
Completed resume	2.81	0	6	Yes

Table 13: Measured local execution of the same full-training SVM search. "New fits" counts candidate-fold fits, excluding the final refit. The resume row reuses a completed search; it is not a partial-interruption timing.

8. Numerical verification and computational experiments

8.1. Protocol and interpretation

The computational experiments examine numerical agreement and resource use, not a second selection of the predictive model. All experiments in this section are executable with the supplied `benchmark.R`. They use the current package and require neither Torch nor Python. Historical development measurements are not substituted for measurements from the reconstructed cohort.

Measurements use Windows 11 x64, R 4.6.0, the default BLAS, and an Intel Core Ultra 7 155H with approximately 15.6 GiB usable physical memory. The full-training experiments retain all 23,246 training drives and the same 5,812 held-out drives as Section 6. SVM fits use cost 0.1. Neural fits use 16 hidden units, 100 epochs, learning rate 0.001, decay 0.0001, and seeds 901, 902 and 903. This fixed budget differs from the tuned neural configuration in Table 6.

Each measurement starts a fresh R process. Fitting seconds exclude process startup, whereas peak working set includes the process, loaded libraries, data, fitting and prediction. The supplied Windows launcher samples private bytes every 100 milliseconds and reads the process peak working set. MiB denotes 2^{20} bytes. These are process measurements, not object sizes or baseline-subtracted allocations. A 900-second or 3,500-MiB private-memory limit is recorded as termination, not as a successful fitting time. On other operating systems, the replication script reports time and explicitly leaves process memory unmeasured. No cross-platform speedup or GPU comparison is claimed.

8.2. Independent numerical audits

The small numerical fixtures are independent checks, not subsets substituted for the application. For both SVM kernels, a 24-row fixture constructs the dense reference Hessian with `Matrix::nearPD()` and compares it with the compact reconstruction. Clarabel and Newton fits are compared through their conditioned objective and centered scores. A constant intercept difference caused by rounded support selection is not evidence of a different ordering, but equal concordance alone is also insufficient evidence of numerical equivalence.

For neural Cox, a 40-row tied-time fixture computes Efron and Breslow partial likelihoods by direct enumeration of risk sets. A central finite-difference calculation checks the analytic gradient, including the weight penalty. The independent calculation does not call the grouped native loss or its gradient. Both event/event and event/censoring ties are retained. Table 14 reports the obtained differences and the declared acceptance tolerances; these are fixture-

Audit	Maximum difference	Tolerance
linear dense/compact Hessian	2.32×10^{-14}	1.00×10^{-9}
linear Clarabel/Newton objective	2.70×10^{-9}	1.00×10^{-5}
linear centered score	4.44×10^{-16}	1.00×10^{-3}
additive dense/compact Hessian	1.08×10^{-14}	1.00×10^{-9}
additive Clarabel/Newton objective	6.54×10^{-12}	1.00×10^{-5}
additive centered score	0	1.00×10^{-3}
efron direct neural loss	4.44×10^{-16}	1.00×10^{-10}
efron finite-difference gradient	5.42×10^{-11}	1.00×10^{-7}
breslow direct neural loss	4.44×10^{-16}	1.00×10^{-10}
breslow finite-difference gradient	6.39×10^{-11}	1.00×10^{-7}

Table 14: Executable independent numerical audits in the current version. The SVM fixture has 24 rows; the neural fixture has 40 rows, two inputs and four hidden units. All calculations use double precision.

Configuration	R	Seconds	MiB	Max MiB	Test C
Neural Cox, Breslow	3	2.00	236.2	237.0	0.8066
Neural Cox, Efron	3	2.00	237.5	237.8	0.8066
Additive SVM, Newton	3	57.56	536.5	542.0	0.6542
Additive SVM, Clarabel	3	259.61	1123.1	1125.2	0.6542
Linear SVM, Clarabel	3	9.37	397.2	405.7	0.7989

Table 15: Current full-training computational measurements. R is the number of fresh processes. Seconds, MiB and test concordance are medians; Max MiB is the largest process peak. SVM costs are fixed at 0.1 and neural runs use the stated fixed epoch budget. No training rows are sampled away.

specific checks, not uniform error bounds.

The full-training additive audit also compares Newton and Clarabel at the same cost. The absolute objective difference is 1.33×10^{-6} and the maximum centered held-out score difference is 2.12×10^{-9}. Every eligible training row is retained in both fits. Exact multiplicity grouping changes the representation of repeated predictor/time/event keys, not the statistical sample or their weights.

8.3. Fresh-process resource measurements

Table 15 reports the median elapsed fitting time, median process peak and largest peak across three fresh processes for all five selected pathways, including additive Clarabel. The SVM rows compare fixed configurations; neural rows compare two explicit tie treatments, not two mathematically identical likelihoods.

From Table 15, we observe lower median fitting time and process peak for additive Newton than for additive Clarabel on this machine. Both pathways now have three fresh-process measurements. Their fixed-cost test concordances are identical, 0.6542, but the objective and score audits above are needed to support numerical agreement. This fixed-cost value must not replace the tuned additive result, 0.734, in Table 6. The native neural Efron and Breslow rows report their separate measured times and peaks; similar concordances do not make their

tie likelihoods identical.

The measurements document the resource requirements of these implementations on one machine. They do not establish universal superiority over external software. In particular, the dense native full-training SVM is not run: a quadratic-storage calculation would not constitute a measured process peak. Similarly, a completed neural epoch budget is not a convergence certificate. The fresh-process outputs retain objective values, numerical violations where applicable, test concordance, process versions and termination status. All three additive Newton workers export the independent certificate from `relative_duality_gap`; the largest relative gap is 3.52×10^{-12}, below the declared 10^{-8} tolerance. The CSV column `relative_gap` is left unavailable for pathways without this specific certificate. Missing Newton certificates or mismatched source, cohort, split, or repetition identifiers cause aggregation to fail. Together with the unit tests, these records distinguish computational feasibility, numerical agreement and statistical prediction.

8.4. Matched software workflows and split sensitivity

The previous examples compare estimators. To assess the interface itself, we additionally fit the same ordinary Cox and Weibull AFT models through **survlearning** and **censored** 0.3.5 with **parsnip** 1.6.1 (Kuhn and Vaughan 2026). Both interfaces use **survival** 3.8-6, the same formula, all 23,246 training drives, and the same 5,812 assessments. Cox uses Efron ties. Three fixed-configuration assessment folds are identical in the two workflows; neither wrapper performs a hyperparameter search in this comparison.

For **censored**, `proportional_hazards()` and `survival_reg(dist = "weibull")` are fitted using the `survival` engine. Its tested `linear_pred` convention points toward longer survival for both models, so its negative is compared with our increasing-risk score. Its survival output is a nested tibble; ours is an observation-by-horizon matrix. Table 16 shows agreement of the centered scores and survival probabilities. Fold scores are also checked with executable tolerances. This agreement validates translation of two shared engines, not superiority of an estimator.

The small formula/parameter/target contract is convenient when replacing learners in one base-style analysis. On the other hand, **censored** integrates with the wider **tidymodels** recipe, workflow, resampling, and tuning infrastructure and supports engine-specific prediction targets that our wrapper deliberately does not reconstruct. A smaller interface therefore trades ecosystem breadth for fewer conventions in this specified workflow. We do not interpret the nested output format or the larger dependency graph as a statistical deficiency, and do not claim a runtime advantage.

For a limited sensitivity assessment, we use split seeds 20261022, 20261028, and 20261029 without retuning. Each split has 23,246 training and 5,812 assessment drives, including 144 assessment failures. Table 17 reports the range and median, not confidence intervals from independent replications. RSF has the highest concordance among these three fixed configurations in each split. These overlapping historical samples do not establish a universal ordering or prospective performance.

Model	Centered score	Survival	C (ours)	C (censored)
Cox, Efron	0	5.9e-15	0.8240	0.8240
Weibull AFT	0	5.6e-17	0.8228	0.8228

Table 16: Matched interface comparison. Score differences are maximum absolute differences after centering; survival differences use 30, 90, and 180 days. Concordances use the same outcome pairs and increasing-risk orientation.

Fixed configuration	Minimum C	Median C	Maximum C
Cox	0.7971	0.8240	0.8254
RSF	0.8475	0.8509	0.8535
Native neural Cox	0.7887	0.8030	0.8127

Table 17: Descriptive concordance across three event-stratified 80/20 splits. Settings are fixed before these fits: ordinary Cox; RSF with 200 trees, node size 15 and three candidate predictors; native neural Cox with 16 hidden units and 100 Efron epochs. Each split retains all 29,058 eligible drives.

9. Reproducibility

The accompanying replication material contains the data, scripts, fitted settings, and software versions needed to reproduce the examples. The entry point `code.R` runs `replication.R`, `comparison.R`, and `benchmark.R` in sequence. The first script reads the cohort, generates the controlled example, performs tuning and prediction, and produces the tables and six figures. It also saves the sample partitions, fold results, selected settings, engine warnings, fitted objects, probability diagnostics, and `sessionInfo()`. These outputs allow the reported comparisons to be traced to the fitted models and assessment samples.

The baseline CSV records the entry-day covariates used in the application. For reconstruction from the original daily snapshots, `prepare_data.R` reads the annual Backblaze archive using base R. Source counts, eligibility criteria, measurement dates, and checksums accompany the data. The statistical analysis therefore requires only the compact baseline file; downloading the annual archive is necessary only to repeat its construction.

The numerical study was run with **survlearning** 0.2.4 and R 4.6.0 on Windows. Its source archive and exact engine versions are supplied with the replication material. The interface described here is version 0.2.5. For compatibility of the reported probability results, we reconstructed the XGBoost Cox baseline with this version using the same fitted engine and all 23,246 training observations. Over 196 assessment horizons, the maximum survival difference was 4.45×10^{-16} and the maximum Brier difference was 3.47×10^{-18}; risk scores, concordance, and integrated Brier scores were unchanged. Computational timings refer to the archived study environment. The manifest `replication-requirements.csv` records the tested versions; installing the latest available engines does not recreate that environment.

The computational script performs the numerical comparisons described in Section 8 and launches each timed fit in a fresh process. On Windows, `benchmark.ps1` measures process memory. Other systems can execute the fits, but their memory usage is reported as unmeasured. Tables are generated from the computed results, and `code.html` provides an executed record of the complete workflow. No component requires Python or Torch. Timings depend

on the machine and concurrent activity.

Software tests complement these empirical comparisons. They check agreement with native engine predictions, event-time ties, extreme finite predictors, fully censored Kaplan–Meier samples, fold-specific transformations, conditional censoring weights, saved-object prediction, and serial/parallel tuning agreement. Reference calculations include explicit weighted pair sums, Weibull conditional probabilities, reverse-Kaplan–Meier ties, and analytic Cox effects. The numerical implementation has been tested on Windows, Linux, and macOS ARM64 and Intel, including an ARM64 build without extended floating-point precision. These tests assess software behavior; they do not establish model adequacy or predictive validity outside the populations considered here.

10. Discussion

In this paper, we presented a common interface for established learning methods with right-censored data. The obtained software separates model fitting, parameter selection, prediction, and concordance evaluation. Its contribution is not a new objective function. Rather, it makes prediction meaning and availability explicit while retaining a small interface for teaching and applied comparisons.

The current implementation has clear restrictions. It handles a single right-censored outcome; the title's broader term "censored data" does not claim left- or interval-censoring support. Competing risks, left truncation, time-dependent covariates, recurrent events, and cure models need distinct contracts. Optional median/mode imputation and grouped/forward-temporal evaluation are available, but nested tuning remains restricted to stratified outer plans. The evaluation interface provides probability diagnostics when native survival curves exist; it does not turn risk-only outputs into probabilities. Marginal reverse-Kaplan–Meier censoring weights also require appropriate censoring assumptions and positive support. Conditional Cox or stratified-KM weights are available, but do not remove model-adequacy and transportability requirements. Conditional evaluation bootstrap intervals exclude model-fitting uncertainty. Interpretation is predictive rather than causal, and remaining-time summaries do not complete an unobserved tail.

The examples should be interpreted within these restrictions. The controlled example exercises all configurations on a common sample but is not a repeated simulation study. The Backblaze example provides a larger censored reliability illustration with verified first-snapshot SMART measurements, but its main comparison uses one historical random split rather than prospective validation. The three fixed-setting sensitivity splits remain historical and overlapping. The subsequent-failure risk set and early-entry probability target are explicit. Linear and additive SVM use every eligible training observation. Native neural Cox avoids an external runtime and handles tied failures, yet retains proportional hazards and a fixed training budget. Its nonconvex loss has no global-optimum guarantee. None of these limitations is removed by a common calling sequence.

Optional engines create practical trade-offs. They avoid installing every engine for a simple analysis and reuse established implementations. On the other hand, engine updates can change numerical behavior, prediction structures, and conventions. Version-specific regression tests and reproducible examples are therefore important for maintaining compatibility as the underlying implementations evolve.

The computational results support the feasibility of the optimized SVM and native neural Cox implementations in the studied setting. They do not establish scaling laws, GPU performance, or superiority across designs. An additive kernel with many retained modes can remain expensive, and its planning guard does not increase physical memory. Further work includes cross-platform profiling, broader repeated data/split benchmarks, additional neural architectures, and independently validated prediction designs. Local parallel and resumable tuning improves control over execution, not the statistical validity of a search. Worker memory duplication, portable formula restrictions, and hard-crash lock recovery remain practical limits. BART, stacking, and other survival outcomes should be added with their own estimation and prediction contracts. For the present scope, **survlearning** connects established survival learning objectives to an inspectable R workflow without requiring every optional engine or a separate neural runtime.

Acknowledgments

The author thanks Melanie Anaís García Fernández for preparing the preliminary presentation on survival learning.

Computational details

The package and replication code use GPL (version 3 or later); engines retain their licenses. Backblaze is credited as the data source, and its data-use conditions apply; the data are not relicensed as package source code.

References

Apley DW, Zhu J (2020). "Visualizing the Effects of Predictor Variables in Black Box Supervised Learning Models." *Journal of the Royal Statistical Society: Series B*, **82**(4), 1059–1086. doi:10.1111/rssb.12377.

Backblaze (2026). *Drive Stats: Hard Drive Reliability and Test Data*. 2013 historical data; accessed 2026-10-20, URL https://www.backblaze.com/cloud-storage/resources/hard-drive-test-data.

Barnwal A, Cho H, Hocking T (2022). "Survival Regression with Accelerated Failure Time Model in XGBoost." *Journal of Computational and Graphical Statistics*, **31**(4), 1292–1302. doi:10.1080/10618600.2022.2067548.

Biganzoli E, Boracchi P, Mariani L, Marubini E (1998). "Feed Forward Neural Networks for the Analysis of Censored Survival Data: A Partial Logistic Regression Approach." *Statistics in Medicine*, **17**(10), 1169–1186.

Binder H, Schumacher M (2008). "Allowing for Mandatory Covariates in Boosting Estimation of Sparse High-Dimensional Survival Models." *BMC Bioinformatics*, **9**, 14. doi:10.1186/1471-2105-9-14.

Breiman L, Friedman JH, Olshen RA, Stone CJ (1984). *Classification and Regression Trees*. Wadsworth.

Chen T, Guestrin C (2016). "XGBoost: A Scalable Tree Boosting System." In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794. doi:10.1145/2939672.2939785.

Chen T, He T, Benesty M, Khotilovich V, Tang Y, Cho H, Chen K, Mitchell R, Cano I, Zhou T, Li M, Xie J, Lin M, Geng Y, Li Y, Yuan J, Cortes D (2026). *xgboost: Extreme Gradient Boosting*. doi:10.32614/CRAN.package.xgboost. R package version 3.2.1.1, URL https://CRAN.R-project.org/package=xgboost.

Cox DR (1972). "Regression Models and Life-Tables." *Journal of the Royal Statistical Society B*, **34**(2), 187–220. doi:10.1111/j.2517-6161.1972.tb00899.x.

Efron B (1977). "The Efficiency of Cox's Likelihood Function for Censored Data." *Journal of the American Statistical Association*, **72**(359), 557–565. doi:10.1080/01621459.1977.10480613.

El Badisy I (2026). ***survalis**: Interpretable Survival Machine Learning Framework*. doi:10.32614/CRAN.package.survalis. R package version 1.0.0, URL https://CRAN.R-project.org/package=survalis.

Fouodo CJK, König IR, Weihs C, Ziegler A, Wright MN (2018). "Support Vector Machines for Survival Analysis with R." *The R Journal*, **10**(1), 412–423. doi:10.32614/RJ-2018-005.

Friedman J, Hastie T, Tibshirani R (2010). "Regularization Paths for Generalized Linear Models via Coordinate Descent." *Journal of Statistical Software*, **33**(1), 1–22. doi:10.18637/jss.v033.i01.

Friedman JH (2001). "Greedy Function Approximation: A Gradient Boosting Machine." *The Annals of Statistics*, **29**(5), 1189–1232. doi:10.1214/aos/1013203451.

Gerds TA, Kattan MW, Schumacher M, Yu C (2013). "Estimating a Time-Dependent Concordance Index for Survival Prediction Models with Covariate Dependent Censoring." *Statistics in Medicine*, **32**(13), 2173–2184. doi:10.1002/sim.5681.

Gerds TA, Schumacher M (2006). "Consistent Estimation of the Expected Brier Score in General Survival Models with Right-Censored Event Times." *Biometrical Journal*, **48**(6), 1029–1040. doi:10.1002/bimj.200610301.

Goulart PJ, Chen Y (2026). "Clarabel: An Interior-Point Solver for Conic Programs with Quadratic Objectives." *Mathematical Programming Computation*. doi:10.1007/s12532-026-00320-7.

Haider H (2026). *MTLR: Survival Prediction with Multi-Task Logistic Regression*. doi:10.32614/CRAN.package.MTLR. R package version 0.2.2, URL https://CRAN.R-project.org/package=MTLR.

Harrell FE, Califf RM, Pryor DB, Lee KL, Rosati RA (1982). "Evaluating the Yield of Medical Tests." *JAMA*, **247**(18), 2543–2546. doi:10.1001/jama.1982.03320430047030.

Harrell FE, Lee KL, Mark DB (1996). "Multivariable Prognostic Models: Issues in Developing Models, Evaluating Assumptions and Adequacy, and Measuring and Reducing Errors." *Statistics in Medicine*, **15**(4), 361–387.

Hester J, Henry L, Muller K, Ushey K, Wickham H, Chang W (2024). *withr: Run Code 'With' Temporarily Modified Global State*. doi:10.32614/CRAN.package.withr. R package version 3.0.2, URL https://CRAN.R-project.org/package=withr.

Higham NJ (2002). "Computing the Nearest Correlation Matrix—A Problem from Finance." *IMA Journal of Numerical Analysis*, **22**(3), 329–343. doi:10.1093/imanum/22.3.329.

Hvitfeldt E, Frick H (2026). ***censored**: **parsnip** Engines for Survival Models*. Accessed 2026-10-20, URL https://censored.tidymodels.org/.

Ishwaran H, Kogalur U (2026). *Fast Unified Random Forests for Survival, Regression, and Classification (RF-SRC)*. R package version 3.6.2, URL https://cran.r-project.org/package=randomForestSRC.

Ishwaran H, Kogalur UB, Blackstone EH, Lauer MS (2008). "Random Survival Forests." *The Annals of Applied Statistics*, **2**(3), 841–860. doi:10.1214/08-AOAS169.

Jackson CH (2016). "**flexsurv**: A Platform for Parametric Survival Modeling in R." *Journal of Statistical Software*, **70**(8), 1–33. doi:10.18637/jss.v070.i08.

Jaeger BC, Welden S, Lenoir K, Pajewski NM (2022). "aorsf: An R package for supervised learning using the oblique random survival forest." *Journal of Open Source Software*, **7**(77), 4705. URL https://doi.org/10.21105/joss.04705.

Jaeger BC, Welden S, Lenoir K, Speiser JL, Segar MW, Pandey A, Pajewski NM (2023). "Accelerated and interpretable oblique random survival forests." *Journal of Computational and Graphical Statistics*. URL https://doi.org/10.1080/10618600.2023.2231048.

Kalbfleisch JD, Prentice RL (2002). *The Statistical Analysis of Failure Time Data*. 2nd edition. John Wiley & Sons. doi:10.1002/9781118032985.

Kaplan EL, Meier P (1958). "Nonparametric Estimation from Incomplete Observations." *Journal of the American Statistical Association*, **53**(282), 457–481. doi:10.1080/01621459.1958.10501452.

Katzman JL, Shaham U, Cloninger A, Bates J, Jiang T, Kluger Y (2018). "DeepSurv: Personalized Treatment Recommender System Using a Cox Proportional Hazards Deep Neural Network." *BMC Medical Research Methodology*, **18**(1), 24. doi:10.1186/s12874-018-0482-1.

Kingma DP, Ba J (2015). "Adam: A Method for Stochastic Optimization." In *International Conference on Learning Representations*. URL https://arxiv.org/abs/1412.6980.

Kuhn M, Vaughan D (2026). *parsnip: A Common API to Modeling and Analysis Functions*. doi:10.32614/CRAN.package.parsnip. R package version 1.6.1, URL https://CRAN.R-project.org/package=parsnip.

Lawless JF (2003). *Statistical Models and Methods for Lifetime Data*. 2nd edition. John Wiley & Sons. doi:10.1002/9781118033005.

Lyu Y (2026). ***SuperSurv**: A Unified Framework for Machine Learning Ensembles in Survival Analysis*. doi:10.32614/CRAN.package.SuperSurv. R package version 0.1.9, URL https://CRAN.R-project.org/package=SuperSurv.

Molstad AJ, Suder PM (2025). *penAFT: Fit the Semiparametric Accelerated Failure Time Model with Elastic Net and Sparse Group Lasso Penalties*. doi:10.32614/CRAN.package.penAFT. R package version 0.3.2, URL https://CRAN.R-project.org/package=penAFT.

Pölsterl S (2020). "**scikit-survival**: A Library for Time-to-Event Analysis Built on Top of **scikit-learn**." *Journal of Machine Learning Research*, **21**(212), 1–6. URL https://jmlr.org/papers/v21/20-729.html.

Pölsterl S, Navab N, Katouzian A (2015). "Fast Training of Support Vector Machines for Survival Analysis." In *Machine Learning and Knowledge Discovery in Databases*, pp. 243–259. Springer-Verlag. doi:10.1007/978-3-319-23525-7_15.

R Core Team (2026). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. doi:10.32614/R.manuals. URL https://www.R-project.org/.

Ridgeway G, Developers G (2026). *gbm: Generalized Boosted Regression Models*. doi:10.32614/CRAN.package.gbm. R package version 2.2.3, URL https://CRAN.R-project.org/package=gbm.

Simon N, Friedman J, Hastie T, Tibshirani R (2011). "Regularization Paths for Cox's Proportional Hazards Model via Coordinate Descent." *Journal of Statistical Software*, **39**(5), 1–13. doi:10.18637/jss.v039.i05.

Sonabend R, Király FJ, Bender A, Bischl B, Lang M (2021). "**mlr3proba**: An R Package for Machine Learning in Survival Analysis." *Bioinformatics*, **37**(17), 2789–2791. doi:10.1093/bioinformatics/btab039.

Suder PM, Molstad AJ (2022). "Scalable Algorithms for Semiparametric Accelerated Failure Time Models in High Dimensions." *Statistics in Medicine*, **41**(6), 933–949. doi:10.1002/sim.9264.

SuperSurv Authors (2026). "Ensemble vs. Best Model Selection." Package vignette, accessed 2026-10-21, URL https://cran.r-project.org/web/packages/SuperSurv/vignettes/supersurv-best.html.

Therneau T, Atkinson B (2026). *rpart: Recursive Partitioning and Regression Trees*. doi:10.32614/CRAN.package.rpart. R package version 4.1.27, URL https://CRAN.R-project.org/package=rpart.

Therneau TM (2026). *A Package for Survival Analysis in R*. R package version 3.8-6, URL https://CRAN.R-project.org/package=survival.

Therneau TM, Grambsch PM (2000). *Modeling Survival Data: Extending the Cox Model*. Springer-Verlag, New York. doi:10.1007/978-1-4757-3294-8.

Venables WN, Ripley BD (2002). *Modern Applied Statistics with S*. Fourth edition. Springer, New York. ISBN 0-387-95457-0, URL https://www.stats.ox.ac.uk/pub/MASS4/.

Wright MN, Ziegler A (2017). "ranger: A Fast Implementation of Random Forests for High Dimensional Data in C++ and R." *Journal of Statistical Software*, **77**(1), 1–17. doi:10.18637/jss.v077.i01.

Yang X, Abraham L, Kim S, Smirnov P, Ruan F, Haibe-Kains B, Tibshirani R (2022). "FastCPH: Efficient Survival Analysis for Neural Networks." doi:10.48550/arXiv.2208.09793. ArXiv:2208.09793.

Yu CN, Greiner R, Lin HC, Baracos V (2011). "Learning Patient-Specific Cancer Survival Distributions as a Sequence of Dependent Regressors." In *Advances in Neural Information Processing Systems*, volume 24. URL https://www.cs.cornell.edu/~cnyu/papers/nips11_survival.pdf.

Zou H, Hastie T (2005). "Regularization and Variable Selection via the Elastic Net." *Journal of the Royal Statistical Society B*, **67**(2), 301–320. doi:10.1111/j.1467-9868.2005.00503.x.

A. Conditioned SVM computation

A.1. The numerical reference and its compact representation

The numerical target is the regression path of **survivalsvm** 0.0.6, including its positive-definite conditioning. This distinction matters: the conditioned program is not asserted to be the unmodified textbook dual of Equation 33. Put $\Delta = \operatorname{diag}(\delta)$ and define

$$B = \begin{pmatrix} K & -M \\ -M & K \end{pmatrix}, \quad M = \tfrac{1}{2}(\Delta K + K\Delta), \quad q = \begin{pmatrix} -Y \\ \Delta Y \end{pmatrix}, \quad a = \begin{pmatrix} -\mathbf{1} \\ \delta \end{pmatrix}. \tag{48}$$

The matrix B is the symmetrized block matrix assembled by the reference. With $H = \operatorname{nearPD}(B)$ under its default unconstrained positive-definite correction, the fitted coefficients solve

$$\min_{u \in \mathbb{R}^{2n}} \ \tfrac{1}{2} u^\top H u + q^\top u, \qquad a^\top u = 0, \qquad 0 \le u_i \le C. \tag{49}$$

Here C denotes the cost, not the censoring time or concordance statistic. The correction follows the projection and eigenvalue-floor conventions of `Matrix::nearPD()` (Higham 2002). The reference's alternative `ipop` path does not apply the same correction and is not treated as an equivalent solver for Equation 49.

Let $Q\Lambda Q^\top$ contain all positive modes retained by the reference relative eigenvalue cutoff $10^{-6}\lambda_{\max}$, and let $\epsilon = 10^{-8}\lambda_{\max}$. The final floor and diagonal rescaling can be represented by

$$H = S\{\epsilon I + Q(\Lambda - \epsilon I)Q^\top\}S = D_0 + LL^\top, \quad D_0 = \epsilon S^2, \quad L = SQ(\Lambda - \epsilon I)^{1/2}. \tag{50}$$

Writing $p_i = (Q\Lambda Q^\top)_{ii}$, the rescaling is $S_{ii}^2 = \max(\epsilon, p_i)/\{p_i + \epsilon(1 - \|Q_{i\cdot}\|^2)\}$. This retains the reference projection, floor, and original-row scaling. It does not replace H by the uncorrected kernel Hessian.

The orthogonal sum/difference transformation reduces the eigensystem of B to those of $K-M$ and $K+M$. For the linear kernel, both lie in the column span of $[X, \Delta X]$. A thin singular-value decomposition reduces the nonzero eigensystems to at most $2p$ dimensions per block. Only numerical null directions below a relative 10^{-12} singular-value threshold are removed; no additional statistical rank parameter is introduced.

For the additive kernel, products can be computed without storing K. For one sorted column $t_1 \le \cdots \le t_n$, define $A_i = \sum_{\ell \le i} v_\ell$ and $B_i^* = \sum_{\ell \le i} t_\ell v_\ell$. Then

$$\sum_{\ell=1}^{n} |t_i - t_\ell| v_\ell = t_i(2A_i - A_n) - (2B_i^* - B_n^*). \tag{51}$$

The identity holds for signed vectors and tied values. It gives exact kernel-vector products up to floating-point arithmetic. Products with $K \pm M$ follow by multiplying by δ and applying the same operator. **RSpectra** computes the largest algebraic eigenvalues adaptively until the converged spectrum includes a mode below the reference cutoff. Residuals and the cutoff boundary are checked; incomplete or ambiguous spectra are rejected. Small or nearly full-rank cases use a dense block eigensystem only when the workspace planning limit permits it. This is a numerical computation of the reference projection, not a Nyström or random-feature approximation.

Identical observations permit another exact reduction. Groups must agree in encoded predictors, observed time, and event indicator. Let P denote group membership, $W = \mathrm{diag}(m_g)$ the multiplicities, $R = PW^{-1/2}$, and $U = \mathrm{blockdiag}(R, R)$. Then $R^\top R = I$. Permutation symmetry and the strictly positive diagonal floor imply that the unique optimum has equal coordinates within each group. Restricting $u = Uz$ gives

$$\min_z \ \tfrac{1}{2} z^\top U^\top H U z + (U^\top q)^\top z, \quad (U^\top a)^\top z = 0, \quad 0 \le z_g \le C\sqrt{m_g}. \tag{52}$$

Weighted kernel operators use $W^{1/2} K_g W^{1/2}$, and the full-row conditioning in Equation 50 is recovered before restriction. Every original coefficient is restored as $u_i = z_{g(i)} / \sqrt{m_{g(i)}}$. No observation is removed from its statistical contribution, and fitting unweighted representatives is not equivalent to Equation 52.

A.2. Solvers, numerical certificates, and tuning

For either the original or multiplicity-restricted coordinates, write the compact Hessian as $D_0 + LL^\top$, with diagonal entries $d_i > 0$ and box bounds b_i. An equivalent sparse quadratic program introduces $v = L^\top z$ and minimizes $\frac{1}{2} z^\top D_0 z + \frac{1}{2} v^\top v + q^\top z$, subject to this identity, the equality constraint, and the boxes. The optional OSQP and Clarabel solvers can use this representation (Goulart and Chen 2026). The native `newton` solver instead eliminates the boxes. Define

$$\psi_i(s) = \min_{0 \le z_i \le b_i} \{\tfrac{1}{2} d_i z_i^2 + s z_i\}, \qquad z_i(s) = \min\{b_i, \max(0, -s/d_i)\}, \tag{53}$$

$$F(\theta, \mu) = \tfrac{1}{2}\|\theta\|^2 - \sum_i \psi_i(q_i + L_{i\cdot}\theta + \mu a_i). \tag{54}$$

For fixed θ, a monotone scalar root solve enforces $a^\top z = 0$. Minimization of the reduced negative dual consequently has r coordinates, where r is the retained rank. Its gradient is

$\theta - L^\top z$. On a cell whose free box coordinates form $\mathcal{A}$, the Hessian is

$$I + L_{\mathcal{A}}^\top D_{\mathcal{A}}^{-1} L_{\mathcal{A}} - \frac{vv^\top}{c}, \quad v = L_{\mathcal{A}}^\top D_{\mathcal{A}}^{-1} a_{\mathcal{A}}, \quad c = a_{\mathcal{A}}^\top D_{\mathcal{A}}^{-1} a_{\mathcal{A}}. \tag{55}$$

The last term is omitted if $c = 0$. The reduced objective is continuously differentiable and piecewise quadratic, not globally twice differentiable at changes in the active set. Analytic derivatives are supplied to `stats::nlm()`, and `stats::uniroot()` enforces the equality. An independent certificate checks the primal–dual objective difference, equality feasibility, and the conservative bound

$$\tfrac{1}{2}\|\theta - L^\top z\|^2 + |\mu a^\top z|. \tag{56}$$

The relative gap must not exceed `tolerance` (default 10^{-8}), and equality error must not exceed $\max(10^{-7}, 10\,\texttt{tolerance})$. Iteration/step-limit failures are rejected. An optimizer return code alone does not certify a valid fit. Likewise, an infeasible reference `quadprog` result cannot win tuning merely because it returns numbers.

All solvers restore original-row coefficients before the reference's five-decimal rounding and support selection. Kernel coefficients are the first block minus δ times the second. The intercept is the mean support residual after rounding, as in the reference. Small coefficient differences can change this support set and introduce a constant score shift. Therefore, raw scores, centered scores, objectives, and concordance are audited separately. Prediction uses support-vector ranges for the additive kernel, matching the reference prediction routine, although training ranges define its training matrix. Out-of-range values are not clipped.

Within each training fold, cost candidates can reuse the same encoded design, kernel factors, and solver workspace. No workspace is shared between folds or with test rows. For four costs and three folds, matrix preparation is reduced from twelve operations to three, plus the selected full-training refit. Linear prediction uses one coefficient vector; additive prediction uses sorted support-prefix sums. Compact storage is proportional to the retained factors rather than a dense $2n$ by $2n$ matrix, but large required ranks can still be expensive. Workspace planning limits are not measured physical-memory caps. Section 8 reports the observed costs and numerical comparisons without claiming a universal speedup.

Affiliation:

Pedro Luiz Ramos
Departamento de Estadística, Facultad de Matemáticas
Pontificia Universidad Católica de Chile
Santiago, Chile
E-mail: pedro.ramos@uc.cl